



RÉPUBLIQUE DU BÉNIN
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ D'ABOMEY-CALAVI

INSTITUT DE FORMATION ET DE
RECHERCHE EN INFORMATIQUE

BP 526 Cotonou Tel : +229 21 14 19 88
<http://www.ifri-uac.net> Courriel : contact@ifri.uac.bj



MÉMOIRE

pour l'obtention du

Diplôme de Master en Informatique

Option : Sécurité Informatique

Présenté par :

Abdel-Faridh MERE NIMA

Architecture de sécurité multicouche pour les systèmes d'intelligence artificielle critiques basée sur l'étude des vulnérabilités et menaces.

Sous la supervision :

Dr (MC) M. Abel KONNON

Membres du jury :

—

Année Académique : 2023-2024

Sommaire

Dédicace	ii
Remerciements	iii
Résumé	iv
Abstract	v
List of Figures	vi
List of Tables	vii
Liste des acronymes	viii
Introduction	1
1 Etat de l’art	4
2 Méthodologie et Architecture	23
3 Implémentation de l’approche proposée	34
4 Résultats et discussion	48
Conclusion	58
Bibliographie	59
Bibliographie	59
Annexe	62
Table des matières	69

Dédicace

Je dédie ce mémoire à :

mon père Idrissou MERE NIMA ;

ma mère Gnou-Ouré DAFIA ;

mes frères et sœurs.

Remerciements

Je tiens à remercier chaleureusement tous ceux qui de près ou de loin ont contribué à la réalisation de ce mémoire ; tout particulièrement :

- Le Professeur Gaston EDAH, Directeur de l’Institut de Formation et de Recherche en Informatique ([IFRI](#)) ;
- Le Docteur (MC) Arnaud AHOUCANDJINO, le Directeur Adjoint de l’Institut de Formation et de Recherche en Informatique ([IFRI](#)) ;
- Le Docteur (MC) M. Abel KONNON, mon maître de mémoire pour avoir accepté d’encadrer ce travail et ses précieux conseils ;
- Dr. Tantchonta M’PO, mon maître de stage pour son soutien et ses différents apports ;
- Tous les professeurs de l’Institut de Formation et de Recherche en Informatique ([IFRI](#)) pour l’enseignement qu’ils ont dispensé ;
- Mon oncle Alassan BABOGOU et ma tante Fati OROU KETEGO pour leur soutien durant ma période de formation ;
- Mes parents, amis et proches qui m’ont soutenus tout le long de ce travail ;

Résumé

L'intégration croissante des systèmes d'intelligence artificielle (IA) dans les environnements critiques expose ces derniers à de nouvelles menaces qui pourraient impacter leur fiabilité, leur sécurité et leur conformité. Les attaques adversariales, l'empoisonnement des données, les atteintes à la confidentialité ainsi que les vulnérabilités de l'infrastructure peuvent compromettre le fonctionnement des modèles d'IA déployés dans des domaines sensibles tels que la sécurité publique, la cybersécurité, la santé ou les transports. Face à ces risques, les mécanismes de protection existants demeurent souvent limités à une menace particulière ou à une couche spécifique du système, sans offrir une vision globale de la sécurité.

Pour pallier cette limitation, ce mémoire propose une architecture de protection holistique et transversale à cinq (05) couches, fondée sur les principes de la défense en profondeur et de l'approche DevSecOps. Cette structure englobe toutes les étapes du cycle de vie des systèmes d'IA, depuis la protection des données d'apprentissage jusqu'à la gouvernance et au suivi des modèles en production. Elle intègre notamment des mécanismes de contrôle des données, l'entraînement adversarial, la protection cryptographique des modèles, la sécurisation des accès, la détection d'anomalies ainsi que le monitoring continu.

Afin de valider cette approche, un prototype a été développé autour d'un système de vision par ordinateur basé sur MobileNetV2 et destiné à la détection d'objets dangereux. L'implémentation de l'architecture a permis de déployer avec succès plusieurs dispositifs de sécurité, notamment la mise en quarantaine des données suspectes, la protection de l'intégrité du modèle par signature numérique, la journalisation sécurisée et auditable des événements ainsi que la supervision des métriques opérationnelles. Les expérimentations démontrent l'efficacité du système, matérialisée par l'interception de plus de 200 images non conformes lors de la phase de contrôle des données, ainsi qu'une résistance accrue face aux attaques adversariales FGSM, marquée par un gain de 18,14 points de précision sous attaque tout en préservant les performances sur les données légitimes.

Ces résultats démontrent qu'une stratégie intégrée alliant sécurité applicative, robustesse algorithmique et supervision continue renforce significativement la résilience des systèmes d'IA critiques face aux menaces actuelles. Ils ouvrent également des perspectives pour l'extension de cette architecture à d'autres types de modèles d'IA et à des environnements critiques à plus grande échelle.

Mots clés : Intelligence artificielle, sécurité des modèles IA, attaques adversariales, systèmes critique, DevSecOps, Défense en profondeur.

Abstract

The increasing integration of artificial intelligence (AI) models into law enforcement systems opens significant opportunities for monitoring, identifying, and preventing criminal acts. However, these models The increasing integration of artificial intelligence systems in critical environments exposes them to new threats that could impact their reliability, security, and compliance. Adversarial attacks, data poisoning, breaches of confidentiality, and infrastructure vulnerabilities can compromise the operation of AI models deployed in sensitive areas such as public safety, cybersecurity, healthcare, and transportation. Faced with these risks, existing protection mechanisms often remain limited to a particular threat or a specific layer of the system, without offering a global view of security.

To overcome this limitation, this thesis proposes a holistic and cross-cutting five-layer protection architecture, based on the principles of defense in depth and the DevSecOps approach. This structure encompasses all stages of the AI system lifecycle, from the protection of training data to the governance and monitoring of models in production. It incorporates data control mechanisms, adversarial training, cryptographic model protection, access security, anomaly detection, and continuous monitoring.

In order to validate this approach, To validate this approach, a prototype was developed around a MobileNetV2-based computer vision system for detecting dangerous objects. The architecture implementation successfully deployed several security features, including quarantining suspicious data, protecting model integrity through digital signatures, secure and auditable event logging, and monitoring operational metrics. Experiments demonstrate the system's effectiveness, evidenced by the interception of over 200 non-compliant images during the data control phase, as well as increased resistance to FGSM adversarial attacks, marked by an 18.14-point gain in accuracy under attack while maintaining performance on clean data.

These results demonstrate that an integrated strategy combining application security, algorithmic robustness, and continuous monitoring significantly enhances the resilience of critical AI against current threats. They also open up possibilities for extending this architecture to other types of AI models and larger-scale critical environments.

Key words: Artificial intelligence, AI model security, adversarial attacks, critical systems, DevSecOps, Defense in depth.

List of Figures

1.1	Décomposition conceptuelle des attaques d'extraction de modèle [24]	13
1.2	Littérature publiée au fil des années sur les attaques et les défenses des modèles de Machine Learning [24]	14
1.3	Répartition des publications par type d'attaques et de défenses [24]	14
2.1	Architecture globale de sécurité à 5 couches	25
3.1	Exemples de la classe "dangerous"	35
3.2	Exemples de la classe "safe"	36
3.3	Pipeline d'entraînement sécurisé	38
3.4	Architecture de production : flux utilisateur, zones de sécurité, et composants	44
4.1	Courbes d'entraînement du modèle sécurisé : loss et accuracy	48
4.2	Comparaison de robustesse FGSM : Baseline vs Sécurisé	50
4.3	Page de connexion avec authentification JWT	51
4.4	Interface web de prédiction : classification Sûr / Dangereux	52
4.5	Dashboard administratif : gestion utilisateurs et audit	52
4.6	Documentation API Swagger : endpoints disponibles	53
4.7	Dashboard Grafana : métriques en temps réel de l'API	53
4.8	Description des couches du système.	62
4.9	Comparaison de la performance baseline : données propres vs adversariales FGSM	65

List of Tables

1.1	Synthèse des principales menaces et attaques contre les modèles d'intelligence artificielle	14
1.2	Synthèse des principales approches de sécurisation des systèmes d'IA	20
3.1	Statistiques du dataset	35
4.1	Résultats d'entraînement du modèle sécurisé	49
4.2	Comparaison baseline vs sécurisé sur données propres	49
4.3	Robustesse FGSM : Baseline vs Sécurisé	50
4.4	Analyse comparative des principales approches de sécurisation des systèmes d'IA . .	54
4.5	Réponse de l'architecture proposée aux limites identifiées dans l'état de l'art	55
4.6	Résultats de l'attaque FGSM sur le modèle baseline	65

Liste des acronymes

ANPR : Automatic Number Plate Recognition

APDP : Autorité de Protection des Données Personnelles [22](#),

API : Application Programming Interface [8](#), [42](#), [44](#),

ART : Adversarial Robustness Toolbox [31](#), [71](#),

BERT : Bidirectional Encoder Representations from Transformers

BIM : Basic Iterative Method [11](#),

C&W : Carlini et Wagner [11](#), [24](#), [27](#), [32](#),

CIA : Confidentialité, Intégrité, Disponibilité [8](#),

CNN : Convolutional Neural Networks

COCO : Common Objects in Context [35](#),

DOS : Denial of Service [9](#),

EU AI Act : Réglementation européenne sur l'intelligence artificielle [21](#), [22](#),

FGSM : Fast Gradient Sign Method [11](#), [15](#), [24](#), [27](#), [32](#), [64](#),

GPT : Generative Pre-trained Transformer

IA : Intelligence Artificielle [iv](#), [1](#), [2](#), [4](#), [8](#), [27](#),

IEEE : Institute of Electrical and Electronics Engineers [31](#),

IFRI : Institut de Formation et de Recherche en Informatique [iii](#),

IRT : Incident Response Team [29](#),

ISO : International Organization for Standardization [21](#), [27](#),

JWT : JSON Web Token [37](#), [43–45](#), [72](#),

LAPI : Lecture Automatique de Plaques d’Immatriculation

ML : Machine Learning [19](#), [70](#),

NIST : National Institute of Standards and Technology [19](#), [70](#),

OCDE : Organisation de coopération et de développement économiques [31](#),

OCR : reconnaissance optique de caractères

PGD : Projected Gradient Descent [15](#), [27](#), [32](#),

RBAC : Role-Based Access Control

REST : Representational State Transfer

RGPD : Règlement Général sur la Protection des Données [21](#), [22](#), [27](#), [29](#), [46](#),

RNN : Recurrent Neural Networks

TRADES : TRadeoff-inspired Adversarial DEfense via Surrogate-loss minimization [16](#),

UNESCO : Organisation des Nations Unies pour l’Éducation, la Science et la Culture [31](#),

WAF : Web Application Firewall [43–45](#), [47](#), [72](#),

Introduction Générale

1. Contexte et justification

L'intelligence artificielle (IA) connaît aujourd'hui une croissance fulgurante et s'impose progressivement comme l'un des principaux moteurs de la transformation numérique des systèmes informatiques modernes. Grâce aux avancées du Machine Learning et du Deep Learning, les modèles d'IA ne sont plus limités à des usages expérimentaux, mais sont désormais intégrés dans des environnements critiques tels que les infrastructures énergétiques, les systèmes financiers, la cybersécurité, la médecine de précision, les systèmes biométriques ou encore les plateformes de mobilité intelligente manipulant tous des données très sensibles ou à forte valeur stratégique. Dans ces contextes, les systèmes d'intelligence artificielle interviennent directement dans l'analyse, la prise de décision et l'automatisation de traitements complexes où la moindre défaillance peut avoir des répercussions systémiques sur la confidentialité, l'intégrité et la fiabilité des services.

Cependant, cette intégration massive des technologies d'IA introduit également de nouvelles problématiques de sécurité. Contrairement aux systèmes logiciels traditionnels dont le comportement est déterministe et relativement prévisible, les systèmes d'intelligence artificielle reposent sur des modèles probabilistes fortement dépendants des données utilisées durant les phases d'apprentissage et d'exploitation. Cette particularité crée une surface d'attaque complexe exposant les modèles à des menaces spécifiques telles que les attaques adversariales, l'empoisonnement des données, l'extraction de modèles ou encore les attaques par inférence. Le rapport publié par le Belfer Center de l'Université Harvard [2] souligne d'ailleurs les domaines les plus vulnérables face à ces attaques.

Plusieurs travaux récents ont démontré que des perturbations imperceptibles peuvent modifier de manière significative les décisions produites par un modèle d'IA. Des études publiées dans la littérature scientifique montrent notamment que certains systèmes de vision par ordinateur peuvent être trompés par des attaques adversariales avec des taux de réussite très élevés. Dans des environnements critiques, de telles compromissions peuvent affecter l'intégrité des décisions automatisées, compromettre la confidentialité des données sensibles et réduire considérablement la fiabilité opérationnelle des systèmes.

Face à ces enjeux, plusieurs mécanismes de protection ont été proposés dans la littérature scientifique, notamment l'entraînement adversarial, la confidentialité différentielle, l'apprentissage fédéré, le chiffrement homomorphe ainsi que les approches DevSecOps et Secure AI Framework de google. Toutefois, la majorité des solutions existantes abordent les problématiques de sécurité de manière isolée, en se concentrant généralement sur une menace spécifique ou une couche particulière du

système.

Dans ce contexte, il devient nécessaire de concevoir des approches globales capables de sécuriser l'ensemble du cycle de vie des systèmes d'intelligence artificielle critiques, depuis la protection des données jusqu'au déploiement et à la supervision continue des modèles en production.

C'est dans cette perspective que s'inscrit le présent mémoire intitulé : « Conception et implémentation d'une architecture de sécurisation des systèmes d'intelligence artificielle critiques basée sur l'étude des vulnérabilités et menaces ».

2. Problématique

L'adoption croissante des systèmes d'IA s'accompagne d'une augmentation significative des menaces visant les modèles, les données et les infrastructures associées. Les vulnérabilités propres aux systèmes d'IA exposent ces derniers à des attaques capables d'altérer le comportement des modèles, de compromettre la confidentialité des données ou de perturber les processus décisionnels automatisés.

Bien que plusieurs mécanismes de protection aient été proposés dans la littérature, les approches existantes restent généralement limitées à des aspects spécifiques tels que la robustesse des modèles, la protection des données ou la sécurisation des infrastructures. Cette fragmentation des mécanismes de défense réduit l'efficacité globale des solutions face à des menaces complexes et évolutives.

Dans ce contexte, la sécurisation des systèmes d'IA traitants des données sensibles ou intégrés dans des environnements critiques nécessite une approche intégrée capable de prendre en compte l'ensemble du cycle de vie des systèmes, depuis les données d'entraînement jusqu'au déploiement et à la supervision des modèles en production. Le présent travail s'inscrit ainsi dans une démarche visant à concevoir et implémenter une architecture de sécurisation multicouche adaptée aux vulnérabilités et menaces affectant les systèmes d'IA déployés dans des environnements très sensibles.

3. Objectifs de l'étude

L'objectif principal de ce travail est de concevoir et d'implémenter une architecture de sécurisation multicouche dédiée aux systèmes d'intelligence artificielle utilisés dans des systèmes critiques, garantissant leur résilience et leur intégrité face aux menaces adverses tout au long de leur cycle de vie.

De manière spécifique, ce travail vise à :

- analyser les principales vulnérabilités et les vecteurs d'attaques ciblant les systèmes d'IA ;
- étudier les techniques de protection existantes ainsi que leurs limites ;
- concevoir une architecture de défense en profondeur intégrant la sécurisation des données, la robustesse des modèles et la protection de l'infrastructure ;
- définir un cadre DevSecOps permettant d'assurer la supervision continue et le maintien en conditions de sécurité des systèmes déployés ;

-
- développer une preuve de concept basée sur un système de vision par ordinateur afin d'évaluer l'efficacité des mécanismes de protection proposés ;
 - analyser les performances et les limites de l'architecture proposée face à différentes attaques adversariales.

4. Organisation du mémoire

Le présent mémoire est structuré en quatre (04) chapitres principaux. Le premier chapitre présente les principaux usages de l'intelligence artificielle dans des systèmes modernes, les principales vulnérabilités et menaces qui les affectent ainsi que les approches de sécurisation existantes. Le deuxième chapitre décrit la méthodologie adoptée, les choix techniques retenus ainsi que l'architecture globale de sécurisation proposée. Le troisième chapitre est consacré à l'implémentation de l'architecture de protection à travers une preuve de concept basée sur un système de vision par ordinateur intégrant plusieurs mécanismes de sécurité. Enfin, le quatrième chapitre met en avant les résultats des expérimentations réalisées, leur analyse comparative ainsi que les limites et perspectives d'amélioration de l'approche proposée.

Etat de l'art

Introduction

Dans un contexte marqué par une incorporation de plus en plus importante des modèles d'intelligence artificielle dans des domaines de forte sensibilité, les questions relatives à la sécurité, à la résilience et à l'éthique se révèlent être des préoccupations prépondérantes. Ce chapitre présente les fondamentaux de l'intelligence artificielle et leurs usages, les vulnérabilités et attaques potentielles auxquelles ils sont susceptibles d'être confrontés, ainsi que les recherches déjà réalisées concernant la sécurité des modèles d'IA déployés dans ces environnements sensibles, ainsi que les normes éthiques et législatives en vigueur.

1.1 L'intelligence artificielle et ses applications

1.1.1 Notion d'intelligence artificielle

L'intelligence artificielle est un secteur de l'informatique qui aspire à doter des systèmes de la faculté de prendre des décisions et d'anticiper, en réalisant des tâches qui auraient nécessité l'intelligence humaine. Selon [31], l'intelligence artificielle est définie comme un algorithme autonome qui peut prendre des décisions et/ou faire des prédictions en imitant l'intelligence humaine. Ce sont des algorithmes dotés de compétences en adaptation, réflexion et apprentissage. Les techniques d'intelligence artificielle nous donnent la possibilité d'automatiser l'extraction de savoirs pour des objectifs prédictifs qui peuvent servir à optimiser la prise de décisions.

Dans le domaine de l'intelligence artificielle (IA), **un modèle d'Intelligence Artificielle** est fondamentalement une structure ou un dispositif conçu pour interpréter et apprendre à partir de données, reproduisant un degré de compréhension et de prise de décisions comparable à celui d'un être humain [8]. C'est une application spécifique des technologies d'intelligence artificielle destinée à résoudre un problème particulier. Ainsi, les modèles d'IA représentent les applications pratiques de ces techniques.

1.1.2 Importance et impact de l'IA dans les systèmes modernes

L'intelligence artificielle (IA) s'est avérée être un pilier technologique incontournable, transformant radicalement les structures socio-économiques actuelles. Autrefois limitée à des applications théoriques, algorithmiques ou à des environnements de laboratoire isolé, elle est maintenant au cœur d'une multitude de secteurs clés. L'IA, en automatisant des procédures décisionnelles complexes et en surpassant occasionnellement les compétences humaines dans le traitement de Big Data, s'est établie comme le catalyseur de l'efficacité opérationnelle contemporaine.

Néanmoins, cette adoption généralisée confère à ces technologies une qualification d'infrastructure essentielle. L'examen approfondi de ses principaux usages souligne non seulement sa capacité de transformation, mais également la fragilité intrinsèque des systèmes qui dépendent d'elle, révélant de nouvelles failles exploitables par des intervenants malintentionnés [15].

1.1.2.1 La Santé et la Médecine de Précision

Le déploiement de l'intelligence artificielle dans le domaine médical progresse à une vitesse vertigineuse, transformant radicalement les méthodes cliniques conventionnelles. Aujourd'hui, l'analyse de l'imagerie médicale, qu'il s'agisse de radiographies, d'IRM ou de scanners, repose indiscutablement sur des modèles basés sur l'apprentissage profond et les réseaux de neurones convolutifs. Ces technologies facilitent l'identification d'anomalies microscopiques, comme des tumeurs naissantes, avec un niveau de précision comparable ou surpassant celui des panels d'experts [12]. En parallèle, l'IA est exploitée pour l'analyse prédictive à partir de dossiers médicaux électroniques complexes, facilitant la prévision de développement de maladies chroniques ou de simuler des interactions moléculaires en vue de trouver de nouveaux traitements. L'acceptation et l'adoption de ces systèmes par la communauté médicale exigent une explicabilité claire de leurs décisions, considérée comme un élément essentiel.

Enjeu de sensibilité et de sécurité :

- Dans ce domaine, l'algorithme est utilisé pour traiter des données extrêmement confidentielles et couvertes par le secret médical.
- Une altération mineure des données entrantes ou une intervention malveillante (comme le détournement de données ou une attaque via l'injection d'exemples conflictuels) pourrait entraîner un diagnostic erroné ou dissimuler une affection grave. Dans ce contexte, le danger ne se limite pas au niveau logiciel : il concerne directement la vie et la sécurité physique des patients.

1.1.2.2 Le Transport et les Véhicules Autonomes

L'émergence des systèmes de transport intelligents et des voitures autonomes dépend entièrement de la convergence de modèles d'intelligence artificielle sophistiqués. Ces architectures utilisent des algorithmes de vision par ordinateur, de fusion de capteurs (LiDAR, radars, caméras) et d'apprentissage par renforcement pour établir une cartographie dynamique de leur environnement en temps réel. L'IA en question se doit de mener à bien une classification instantanée des éléments environnants (piétons, panneaux de signalisation, autres véhicules), estimer leurs futurs mouvements et orchestrer des décisions relatives à la navigation et au freinage en l'espace de quelques millisecondes.

Enjeu de sensibilité et de sécurité :

- La spécificité de ce secteur en fait l'un des cas les plus cruciaux concernant la sécurité des systèmes intégrés. Un bug algorithmique, une inaccessibilité du modèle ou une attaque par perturbation physique, comme l'application d'éléments géométriques ou d'autocollants conçus pour induire en erreur les classificateurs visuels, peut mener à des accidents fatals ou à paralyser des flottes de transport logistique, mettant en péril la sécurité routière et publique à grande échelle.

1.1.2.3 Le Secteur Financier, Bancaire et l'Économie Globale

Les institutions financières et les marchés boursiers à l'échelle mondiale ont basé leurs approches de gestion des risques sur des systèmes d'intelligence artificielle profondément automatisés. Ces algorithmes étudient des flux incessants de transactions financières mondiales afin de repérer, grâce à des méthodes de détection d'anomalies, des schémas de fraude par carte de crédit, des transactions de blanchiment de capitaux ou des attaques informatiques ciblant les infrastructures bancaires. De surcroît, le trading algorithmique recourt à l'IA pour juger la capacité de remboursement des débiteurs et mener des transactions boursières à haute fréquence selon les variations économiques et géopolitiques [19].

Enjeu de sensibilité et de sécurité :

- L'intégrité de ces modèles est directement associée à la stabilité macroéconomique.
- Un accès malintentionné aux données d'entraînement d'un modèle de notation de crédit ou le piratage d'une interface de programmation (API) d'inférence boursière pourrait mener à des décisions financières inexactes à grande échelle, entraînant un désastre financier majeur, des dépôts de bilan d'entreprises ou des effondrements boursiers créés de toutes pièces.

1.1.2.4 La Relation Client, les Services et la Gestion des Flux d'Information

Grâce aux progrès significatifs dans le domaine du traitement automatique du langage naturel (TALN) et des grands modèles de langage (LLM), les chatbots et les assistants virtuels remodelent l'interaction humaine avec les divers services. Que ce soit dans le domaine bancaire, des télécommunications ou des services publics, ces systèmes traitent quotidiennement des millions de demandes. Ils assurent une présence constante et une personnalisation précise des réponses en extrayant habilement des données à partir des bases de connaissances d'entreprise.

Enjeu de sensibilité et de sécurité :

- Ces interfaces sont à l'avant-garde face aux utilisateurs finaux et gèrent un volume considérable de données personnelles nominatives.
- Elles sont particulièrement exposées aux attaques par injection d'instructions (Prompt Injection), lorsqu'un individu aux intentions malveillantes incite l'IA à franchir ses limites sécuritaires afin de révéler des données sensibles, pomper des bases de données clients ou exécuter des opérations non réglementées sur le système d'information sous-jacent.

1.1.2.5 La Gestion des Infrastructures Critiques et de l'Énergie

Les réseaux électriques intelligents (Smart Grids), les centrales de production d'électricité (nucléaire, hydraulique, thermique) ainsi que les usines de traitement et de fourniture d'eau potable se servent actuellement de l'intelligence artificielle pour améliorer leur efficacité. Les algorithmes prédisent la consommation d'énergie en direct pour maintenir l'équilibre sur le réseau, anticipent les défaillances

des équipements lourds via la maintenance prédictive, et régulent les flux de distribution de fluides pour prévenir les surpressions ou les pénuries d'approvisionnement.

Enjeu de sensibilité et de sécurité :

- Ce secteur incarne le niveau de criticité le plus élevé au sein de l'industrie.
- Si les systèmes d'IA de supervision ou leurs seuils d'alerte algorithmique sont compromis ou manipulés, un attaquant étatique ou un cybercriminel pourrait causer un black-out électrique à l'échelle régionale, interrompre l'approvisionnement en eau potable d'une grande ville ou détruire des machines industrielles de grande taille, ce qui représenterait une menace directe pour la sécurité nationale et la continuité de la vie quotidienne.

1.1.2.6 La Sécurité Publique, la Télésurveillance et l'Analyse en Milieu Sensible

Dans les environnements hautement sécurisés, comme les espaces aéroportuaires, les installations industrielles classées, les infrastructures gouvernementales ou encore les points de contrôle aux frontières, on observe une large utilisation de l'intelligence artificielle liée à la vision par ordinateur. Elle permet d'automatiser l'identification d'anomalies, la reconnaissance de comportements suspects, la détection des intrusions de périmètre ou encore la localisation d'objets menaçants (armes, paquets laissés sans surveillance). Ces dispositifs fonctionnent comme des amplificateurs de force pour les agents de sécurité en combinant simultanément des milliers de flux vidéo.

Enjeu de sensibilité et de sécurité :

- La résilience de ces modèles aux tentatives de déviation doit être totale. Si un adversaire réussit à tirer parti d'une vulnérabilité de type "tache aveugle" (blind spot) ou à utiliser un motif adverse physique, comme un vêtement ou un insigne imprimé conçu pour duper l'IA de détection de silhouettes [30], il pourrait se glisser clandestinement dans un secteur d'importance cruciale. Dans ce contexte, la sécurité physique des infrastructures et des personnes dépend ici directement de la robustesse mathématique de l'IA face aux attaques adverses.

1.1.2.7 Le Domaine Juridique et l'Aide à la Décision Institutionnelle

Des systèmes d'IA s'implémentent progressivement dans le système judiciaire et les administrations publiques à travers des outils de justice prédictive et d'aide à la décision. Ces systèmes examinent d'énormes quantités de jurisprudence pour apprécier les probabilités de succès d'une procédure, estimer le montant des compensations ou contribuer à déterminer le risque de réitération dans certains systèmes de justice pénale. Ils sont aussi utilisés pour automatiser la distribution des aides sociales ou pour examiner des dossiers de demande d'immigration, en s'appuyant sur l'analyse de profils de données [19].

Enjeu de sensibilité et de sécurité :

- Ce champ d'application concerne directement les libertés personnelles, les droits fondamentaux et à l'éthique républicaine.
- Les modèles d'IA ont tendance à refléter et à exagérer les préjugés discriminatoires (qu'ils soient socio-économiques, raciaux ou liés au genre) qui existent dans les données d'entraînement historiques.

- Si ces mécanismes d'évaluation ou de prédiction sont entachés de corruption ou mal audités, ils peuvent consolider des iniquités systémiques, priver injustement les citoyens de leurs droits ou fausser des jugements judiciaires, entraînant une grave fracture de confiance entre les citoyens et les institutions de l'État.

Cette interpénétration croissante et irréversible de l'IA au sein des fonctions les plus vitales de notre société actuelle suscite des préoccupations légitimes et profondes. Au fur et à mesure que l'autonomie conférée à ces modèles s'accroît, les limites conventionnelles de la cybersécurité se transforment. La fiabilité d'une infrastructure ne se fonde plus uniquement sur l'efficacité de ses pare-feu ou sur la sécurisation de ses échanges, mais aussi sur la résistance inhérente de ses algorithmes d'apprentissage face à toute tentative de manipulation.

Ainsi, les impératifs relatifs à la sécurité des pipelines d'apprentissage, la protection des données d'apprentissage et la robustesse des modèles tout au long de leur cycle opérationnel deviennent des enjeux majeurs pour la recherche scientifique et la gouvernance technologique contemporaine.

1.1.3 Notion de systèmes d'IA critiques

Dans le cadre de cette étude, les systèmes d'intelligence artificielle critiques font référence aux systèmes informatiques intégrant des capacités d'intelligence artificielle l'IA et déployés dans des environnements où une défaillance, une erreur de prédiction, une manipulation malveillante ou une indisponibilité du système peut entraîner des répercussions majeures en termes de sécurité des personnes, de continuité des services vitaux, de protection des données sensibles ou de fonctionnement d'infrastructures stratégiques. Ces dispositifs sont particulièrement employés dans des secteurs tels que la santé, le transport intelligent, la sécurité publique, les infrastructures essentielles et certains services gouvernementaux. Leur caractère critique résulte moins de la technologie mise en œuvre que de l'impact potentiel de leur dysfonctionnement sur les individus, les organisations ou la société.

1.2 Sécurité et intelligence artificielle

L'expansion des systèmes d'intelligence artificielle dans des secteurs vitaux tels que la sécurité publique, la justice prédictive et la surveillance suscite des problématiques cruciales en termes de sécurité informatique. L'application des normes traditionnelles de cybersécurité aux modèles d'intelligence artificielle se base surtout sur le triptyque **CIA(Confidentialité, Intégrité, Disponibilité)** pour assurer leur fiabilité et leur sécurité.

- **Confidentialité** : La confidentialité vise à protéger les données d'apprentissage et les modèles contre toute accès non autorisé. Les modèles d'IA manipulent fréquemment des informations sensibles (dossiers médicaux, images faciales, opérations bancaires, et bien d'autres), ce qui les rend vulnérables aux violations de données. Des attaques comme **l'extraction de modèles** permettent aux attaquants de reconstruire un modèle en interrogeant son API, exposant ainsi des informations confidentielles.

- **Intégrité** : L'objectif de l'intégrité est d'éviter toute altération non autorisée du modèle ou de ses données. Les **attaques adversariales** tirent parti de cette vulnérabilité en modifiant légèrement les entrées d'un modèle afin de provoquer des erreurs dans ses prédictions. Par exemple, un algorithme d'identification d'image peut être induit en erreur par des altérations subtiles sur une image, menant à des classifications incorrectes.
- **Disponibilité** : La disponibilité assure que les modèles d'IA demeurent accessibles et fonctionnels en même en cas d'attaques ou de dysfonctionnements. Les infrastructures IA sont visées par des attaques de **Déni de Service (DOS)**, qui surchargent les ressources, provoquant ainsi un ralentissement ou une interruption de leur fonctionnement. La protection des modèles IA englobe la supervision de tout le processus d'apprentissage automatique, depuis la collecte des données jusqu'au déploiement du modèle.

1.3 Typologie des vulnérabilités des modèles d'IA

Les modèles d'intelligence artificielle, notamment les réseaux de neurones profonds, ont des faiblesses intrinsèques liées à leur conception, à la qualité des données employées pour les entraîner et aux conditions dans lesquelles ils sont exploités. Cette section présente une classification des principales vulnérabilités exploitées dans les attaques visant ces systèmes. L'expansion des modèles d'intelligence artificielle a révélé plusieurs faiblesses que des attaquants ont pu exploiter pour entacher leur fiabilité et leur sûreté. On peut classer ces vulnérabilités en différentes catégories :

1.3.1 Vulnérabilités structurelles et inhérentes aux modèles IA

Les modèles d'IA sont sensibles à des failles inhérentes à leur conception et à leur entraînement :

- **Sur-apprentissage (Overfitting)** : Quand le modèle se conforme trop étroitement aux données d'entraînement, il perd son efficacité avec de nouvelles données. Le modèle n'extrait pas les règles implicites mais apprend par cœur les données d'entraînement. Il est à nouveau incapable de généraliser efficacement ses prédictions à de nouvelles entrées [3].
- **Sensibilité aux données bruitées** : Les modèles peuvent être aisément induits en erreur par des données bruitées ou adversariales [15]. Cela implique qu'une légère altération des données d'entrée peut provoquer des fautes notables dans les prédictions.
- **Exploitabilité des gradients** : Les modèles neuronaux, en particulier les réseaux profonds, sont susceptibles d'être ciblés par des attaques qui exploitent les gradients pour produire des entrées malveillantes. D'après [15], ces attaques exploitent les gradients de la fonction de coût afin de repérer les légers changements susceptibles d'altérer considérablement la classification du modèle.

1.3.2 Vulnérabilités liées aux données d'entraînement

Les données constituent un vecteur clé des vulnérabilités des modèles d'IA :

1.3.2.1 Empoisonnement des données (Data Poisoning) :

Il s'agit de l'introduction intentionnelle de données corrompues dans les données d'entraînement d'un modèle de machine learning lors de son processus d'apprentissage. Les modèles d'IA dépendent fortement de la qualité et de l'intégrité des données d'entraînement, si ces données contiennent des anomalies, le modèle peut en hériter, réduisant ainsi sa fiabilité, altérant ses performances et biaisant ses décisions. [24].

1.3.2.2 Biais et discrimination algorithmique :

Le biais algorithmique est simplement un écart systématique dans la sortie, la performance ou l'impact de l'algorithme, par rapport à une norme ou à un standard [13]. Ces biais conduisent souvent à des résultats injustes ou contradictoires pour certains groupes ou personnes. [13, 21] identifient de nombreux biais observés ou relevés dans les systèmes d'intelligence artificielle et robotiques, y compris ceux présents dans les applications de reconnaissance faciale, de reconnaissance vocale ainsi que dans des moteurs de recherche. Les biais algorithmiques peuvent trouver leurs origines à divers niveaux. Selon [13] et [21], les biais peuvent être créés : soit lors de la spécification du problème, soit dans les données, soit lors de la modélisation et la validation, soit dans le déploiement, soit lors des interactions avec l'utilisateur.

1.3.2.3 Fuites d'informations sensibles :

Les modèles d'IA ont la capacité de retenir et de révéler, sans le vouloir, des informations sensibles provenant de leur jeu de données d'apprentissage. Ce danger devient particulièrement alarmant lorsque les modèles sont disponibles publiquement et entraînés à partir de données privées, surtout dans le contexte des réseaux neuronaux employés pour la classification ou la prédiction. Comme le souligne [11], les modèles d'IA, spécifiquement les générateurs de séquences, risquent de retenir des informations sensibles telles que des numéros de carte bancaire ou des données personnelles uniques contenues dans leurs bases d'apprentissage. Ceci se produit même si ces informations ne reflètent pas la tâche principale de manière représentative.

1.3.3 Vulnérabilités liées à l'inférence et à l'exploitation des modèles

Après leur déploiement, les modèles d'intelligence artificielle exposent naturellement certaines failles, même sans action malveillante explicite. Ces vulnérabilités proviennent du comportement prédictif du modèle et de la manière dont il traite les entrées. Elles peuvent être exploitées de manière passive pour obtenir des informations sensibles sans altérer directement l'état du modèle.

1.3.3.1 Vulnérabilités par inférence de modèle :

Les modèles peuvent révéler des informations sensibles par la simple observation de leurs sorties. Deux principales formes de vulnérabilités sont reconnues :

- **Inférence d'appartenance** : En analysant les scores de prédiction, on peut deviner si un exemple donné appartenait ou non au jeu d'entraînement. Les modèles ayant une forte mémorisation tendent à répondre avec des niveaux de confiance plus haut pour les exemples vus.

- **Inférence de modèle** : En interrogeant un modèle de façon répétée, un adversaire peut extraire des caractéristiques internes, comme l'architecture, les fonctions d'activation, ou les régularisations utilisées.

Ces vulnérabilités, même sans attaque explicite, exposent la confidentialité des données et la propriété intellectuelle du modèle.

1.3.3.2 Vulnérabilités par extraction de modèle

Une autre vulnérabilité importante après le déploiement d'un modèle d'intelligence artificielle concerne la possibilité pour un adversaire de **reconstruire** ou **d'imiter** le comportement du modèle cible, uniquement en l'interrogeant de manière répétée. Ce phénomène est connu sous le nom de **vulnérabilité à l'extraction de modèle**. Lorsqu'un modèle est exposé via une API publique ou un service accessible à distance (par exemple un modèle de reconnaissance faciale ou de détection d'anomalies), un attaquant peut soumettre de nombreuses requêtes en fournissant des entrées variées et collecter systématiquement les sorties produites [24]. À partir de ces paires (*entrée – prédiction*), il peut alors entraîner son propre **modèle substitut** capable de reproduire fidèlement les décisions du modèle cible, sans jamais y avoir eu un accès direct.

1.4 Classification des attaques contre les modèles d'IA

À partir des vulnérabilités identifiées, diverses attaques ont été développées afin de compromettre l'intégrité, la confidentialité, la robustesse ou la disponibilité des systèmes IA. Elles peuvent survenir pendant l'entraînement, lors de l'inférence ou via une interaction distante avec le modèle. Cette section présente une classification de ces attaques.

1.4.1 Attaques adversariales

Ce sont des modifications intentionnelles des entrées (images, données) visant à induire en erreur les modèles d'intelligence artificielle (IA), notamment ceux basés sur l'apprentissage profond. Ces offensives tirent parti des failles présentes dans les modèles afin de modifier leurs prédictions, et ce même si les perturbations appliquées à l'entrée demeurent invisibles à l'œil humain. [24] et [7] citent différentes techniques couramment employées pour produire des perturbations adversariales :

- **Fast Gradient Sign Method (FGSM)** : Cette technique induit rapidement des perturbations en utilisant les gradients des fonctions de coût des modèles.
- **Basic Iterative Method (BIM)** : Une variante itérative du FGSM qui perfectionne les perturbations à chaque étape.
- **Attaques universelles** : Ces perturbations peuvent induire en erreur le modèle sur toute sorte d'image, les rendant indépendantes du contenu de l'image.
- **Attaques spécifiques comme celles de Carlini et Wagner (C&W)** : Elles génèrent des perturbations presque indiscernables mais extrêmement efficaces.

Ces attaques mettent en lumière les failles de robustesse des modèles d'IA, soulevant des interrogations sur leur crédibilité dans des applications cruciales telles que la reconnaissance faciale, les véhicules autonomes ou encore la sécurité biométrique.

1.4.2 Attaques sur les données d'entraînement

Les attaques visant la phase d'apprentissage tirent profit du besoin d'un modèle IA de traiter d'importants volumes de données. En manipulant délibérément ces ensembles de données, un attaquant peut orienter discrètement ou de manière considérable les décisions du modèle.

1.4.2.1 Poisoning Attacks (attaques d'empoisonnement)

Ces attaques visent à corrompre l'ensemble d'entraînement afin d'affecter durablement la qualité du modèle.

- **Availability poisoning (empoisonnement de disponibilité)** : l'attaquant injecte en vrac des exemples erronés (étiquetage aléatoire, données fortement bruitées ou extrêmes) pour dégrader globalement la performance. Par exemple, en falsifiant plusieurs images d'un chat pour qu'elles ressemblent subtilement à des chiens, on observe une chute significative de l'exactitude moyenne, parfois jusqu'à diviser par deux la précision initiale [10].
- **Integrity poisoning (empoisonnement d'intégrité)**

Ici, l'objectif est plus ciblé : corrompre uniquement un ou quelques points spéciaux (une ou quelques classes) sans impacter le reste du modèle. L'attaquant modifie des échantillons situés près de la frontière de décision dans l'espace des caractéristiques, de façon à ce que ces cas précis soient systématiquement mal classés, tout en maintenant une performance normale sur les autres exemples [29].

1.4.2.2 Backdoor Attacks (attaques par porte dérobée)

Plutôt que de perturber en masse, les attaques par **porte dérobée** (backdoors) sont conçues pour rester invisibles dans la majorité des cas, n'activant le comportement malveillant que lorsqu'un **trigger** spécifique est présent. En l'absence du trigger, le modèle se comporte normalement, rendant l'attaque quasi indétectable jusqu'à l'activation réelle. Cette dualité rend la backdoor extrêmement difficile à détecter, car sans trigger, le modèle fonctionne parfaitement.

1.4.3 Attaques sur la confidentialité et la robustesse des modèles

Les vulnérabilités passives évoquées précédemment peuvent être mises à profit par des attaques actives dans le but d'obtenir des informations, de reproduire un modèle ou de perturber son fonctionnement. Ces offensives compromettent la confidentialité des utilisateurs, la protection de la propriété intellectuelle ainsi que l'intégrité opérationnelle du modèle.

1.4.3.1 Membership Inference Attack (MIA)

Les attaques par inférence de l'appartenance (Membership Inference Attack, ou MIA) explorent la capacité des modèles d'apprentissage automatique à révéler des informations sur les données individuelles utilisées lors de leur entraînement. L'objectif est de déterminer si un enregistrement donné fait partie du jeu de données d'entraînement d'un modèle, en se basant uniquement sur l'accès en boîte noire au modèle, c'est-à-dire en utilisant les prédictions fournies par le modèle sur cet enregistrement [28]. Ces attaques ont pour impact de graves violations de la confidentialité des données personnelles, notamment en santé, finance, ou justice.

1.4.3.2 Model Extraction Attacks :

Les attaques par extraction de modèle, également appelées **model stealing**, sont des techniques visant à copier ou à reproduire un modèle d'apprentissage automatique au détriment du propriétaire légitime du modèle. À travers une interaction continue avec l'API publique du modèle, un adversaire cherche à obtenir des informations précises sur le modèle cible, telles que ses hyperparamètres d'entraînement, son architecture ou ses paramètres appris. Elles peuvent également viser à reproduire le comportement du modèle cible pour des tâches similaires.

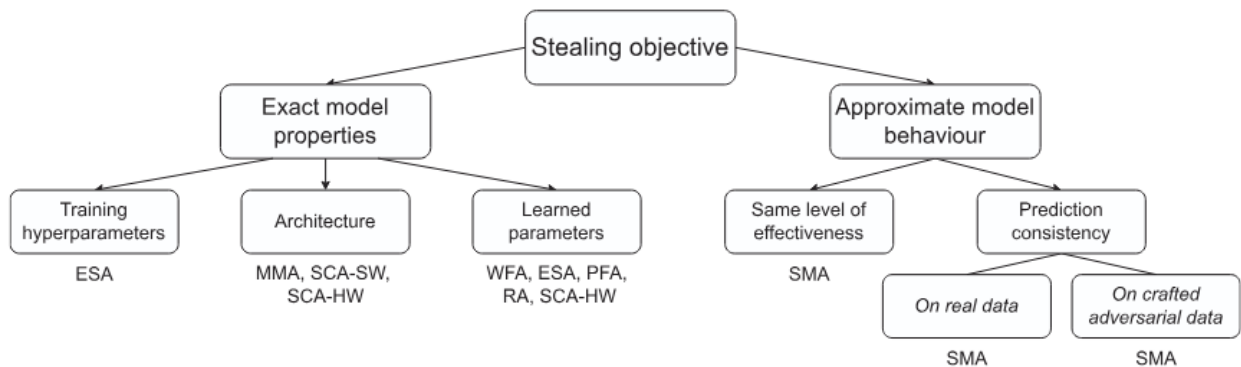


FIGURE 1.1 – Décomposition conceptuelle des attaques d'extraction de modèle [24]

1.4.3.3 Illustration de l'évolution des menaces et des défenses

Pour mieux comprendre l'évolution de ces menaces, la figure ?? issue du travail de [24] offre une vue d'ensemble précieuse. Elle montre une **augmentation significative du nombre de publications scientifiques** portant sur les attaques contre les modèles d'apprentissage automatique, en particulier à partir de 2016. Si les publications sur les attaques dominaient au début, les travaux sur les **mécanismes de défense** ont connu une croissance notable dès 2018. On y distingue notamment : la **prévalence des attaques par entraînement de modèle substitutif**, qui consistent à interroger un modèle cible pour entraîner un clone ; et l'émergence de **techniques de défense** comme la perturbation de sortie et la surveillance du comportement du modèle

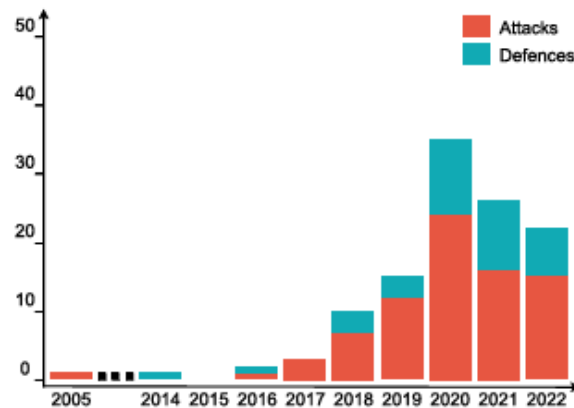
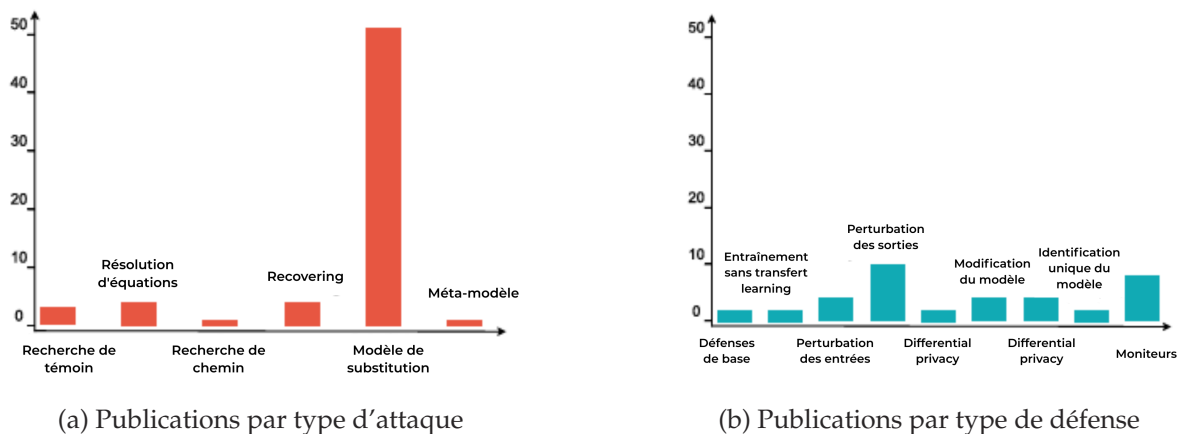


FIGURE 1.2 – Littérature publiée au fil des années sur les attaques et les défenses des modèles de Machine Learning [24]



(a) Publications par type d'attaque

(b) Publications par type de défense

FIGURE 1.3 – Répartition des publications par type d'attaques et de défenses [24]

Cette progression illustre l'importance croissante accordée à la **sécurité spécifique des modèles IA** dans la communauté scientifique, et souligne la nécessité de proposer des approches robustes, intégrées et auditées.

TABLE 1.1 – Synthèse des principales menaces et attaques contre les modèles d'intelligence artificielle

Type d'attaque	Phase concernée	Impact sur le système
Attaques adversariales (FGSM, PGD, etc)	Inférence	Erreurs de classification, perte de fiabilité
Empoisonnement des données	Entraînement	Biais du modèle, dégradation des performances
Extraction de modèle	Exploitation	Vol de propriété intellectuelle
Attaques par inférence	Exploitation	Atteinte à la confidentialité des données

1.5 Analyse des travaux existants sur la protection des systèmes IA

Cette section offre un aperçu des méthodes actuelles pour la protection des modèles d'IA, spécifiquement en ce qui concerne les attaques adversariales et les menaces à la confidentialité.

1.5.1 Approches de défense adversariale

Depuis les premières recherches de [15] concernant le FGSM, les méthodes de défense contre les attaques adversariales ont considérablement progressé. On distingue :

1.5.1.1 Entraînement adversarial

L'entraînement adversarial, popularisé par [20], redéfinit le paradigme d'apprentissage standard en incorporant des exemples altérés directement dans le processus d'optimisation. Au lieu de se focaliser exclusivement sur la réduction du risque empirique lié à la distribution des données propres comme le fait l'apprentissage conventionnel, cette méthode envisage le processus comme un problème de Min-Max (Selle-Minimax). L'objectif est d'identifier les paramètres du modèle qui diminuent la perte maximale qu'un assaillant pourrait causer par le biais d'une perturbation bornée.

Formalisation Mathématique : L'optimisation robuste face à un attaquant opérant sous une contrainte de norme ℓ_p (généralement ℓ_∞ ou ℓ_2) se formalise par le problème d'optimisation Min-Max suivant :

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{\|\delta\|_p \leq \epsilon} \mathcal{L}(f_{\theta}(x + \delta), y) \right] \quad (1.1)$$

Où :

- θ représente les paramètres (poids et biais) du réseau de neurones profonds.
- $\mathcal{D}$ correspond à la distribution empirique des données d'entraînement.
- δ est le vecteur de perturbation adverse injecté dans l'entrée propre x .
- ϵ définit le budget d'attaque maximal (rayon de la boule de perturbation).
- $\mathcal{L}$ désigne la fonction de perte (généralement l'entropie croisée).

Résultats Opérationnels et Robustesse : L'intégration de la PGD au cours de la phase d'apprentissage confère au modèle une augmentation drastique de sa précision robuste (*Robust Accuracy*). Face à des attaques de type FGSM ou PGD à faible nombre d'itérations, le modèle conserve un taux de classification correct acceptable en comparaison avec un modèle standard dont les performances s'effondrent à 0%. Cette méthode force le réseau à apprendre des caractéristiques robustes et sémantiquement significatives de l'image (comme les formes globales) plutôt que des motifs statistiques de haute fréquence imperceptibles mais hautement vulnérables.

Limites : Malgré sa robustesse théorique, l'entraînement adversarial souffre de trois limites majeures :

1. **Le Dilemme Précision-Robustesse :** On observe une dégradation systématique de la précision sur les données saines (*Clean Accuracy*). Sécuriser le modèle contre les pires scénarios réduit sa capacité de généralisation en conditions nominales.
2. **Le Surapprentissage Adversarial (*Overfitting Adversarial*) :** Au-delà d'un certain nombre d'époques, le modèle surapprend la nature spécifique des exemples PGD générés pendant le train, perdant sa capacité de généralisation face à d'autres distributions d'attaques.
3. **Coût Calculatoire Prohibitif :** Multiplier par K étapes de gradient la boucle interne à chaque batch d'entraînement augmente le temps de calcul d'un facteur 10 à 30, rendant le passage à l'échelle difficile sur de très grands jeux de données sans infrastructures massives.

1.5.1.2 TRADES et défenses basées sur la régularisation

Pour pallier le compromis rigide entre la précision nominale et la robustesse, [32] ont introduit **TRADES** (*TRadeoff-Inspired Adversarial Defense via Surrogate-Loss Minimization*). Le principe fondamental consiste à décomposer mathématiquement l'erreur robuste en deux composantes distinctes : l'erreur empirique sur les données propres et l'erreur de robustesse résultant de la sensibilité de la frontière de décision aux perturbations adversariales. Contrairement à l'Adversarial Training classique, qui cherche à maximiser directement la perte de classification, TRADES introduit un terme de régularisation fondé sur la divergence de Kullback-Leibler (KL). Ce terme contraint les prédictions associées aux exemples adversariaux à demeurer proches de celles obtenues sur les exemples non perturbés, favorisant ainsi un meilleur équilibre entre précision et robustesse.

Apport Spécifique : **TRADES** permet un ajustement précis de la surface décisionnelle du modèle en lissant les gradients locaux de la fonction de décision. Les résultats d'expérimentation actuels montrent que **TRADES** présente de meilleures performances que l'entraînement adversarial de Madry face à des attaques sophistiquées telles que l'AutoAttack, tout en gardant une « Clean Accuracy » supérieure pour un degré de robustesse équivalent. En imposant au réseau de maintenir une stabilité au sein de la sphère de perturbation, cette mesure défensive permet de repousser de manière efficace la limite de décision bien au-delà des points de données réels.

Limites Techniques : Le problème principal est une sensibilité excessive à la configuration de l'hyperparamètre β . Une mauvaise configuration peut soit compromettre la protection, soit réduire la performance globale de l'application sous les niveaux acceptables en environnement critique. Par ailleurs, l'optimisation de la divergence KL pendant la boucle interne demeure complexe et nécessite un grand nombre d'itérations PGD, ce qui impose une lourde charge de calcul durant la phase de développement.

1.5.1.3 Distillation défensive et transformations d'entrée

La **Distillation Défensive** (*Defensive Distillation*), proposée par [25], adapte au domaine de la sécurité des modèles d'intelligence artificielle le mécanisme de transfert de connaissances introduit par

Hinton et ses collaborateurs. Cette approche consiste à entraîner dans un premier temps un modèle *enseignant* (*Teacher*), puis à utiliser les probabilités de sortie lissées, appelées *soft labels*, générées par ce modèle pour entraîner un second modèle *élève* (*Student*) possédant généralement la même architecture.

Le principe repose sur l'introduction d'un paramètre de température T élevé dans la fonction Softmax durant l'apprentissage. Cette température produit des distributions de probabilités moins abruptes et vise à réduire la sensibilité du modèle aux faibles perturbations appliquées aux données d'entrée, en atténuant les gradients exploités par les attaques adversariales.

Parallèlement, les **transformations d'entrée** constituent une famille de mécanismes défensifs reposant sur l'application de prétraitements déterministes ou stochastiques avant l'inférence. Ces transformations peuvent prendre différentes formes, telles que la compression JPEG, la réduction de la profondeur des couleurs, le redimensionnement d'image ou encore la randomisation de certains pixels. Leur objectif est de supprimer ou d'atténuer les perturbations adversariales introduites dans les données d'entrée avant leur traitement par le modèle.

Résultats Opérationnels : Lors de son introduction, la distillation défensive a montré des résultats prometteurs contre plusieurs attaques de première génération, notamment les attaques L-BFGS et FGSM. Les expérimentations initiales ont révélé une réduction significative du taux de réussite de ces attaques, suggérant une amélioration importante de la robustesse du modèle. Les transformations d'entrée présentent également l'avantage d'être relativement simples à mettre en œuvre dans une chaîne de traitement existante. Elles peuvent être intégrées dans des pipelines applicatifs ou des environnements conteneurisés sans nécessiter de réentraînement complet du modèle, ce qui réduit leur coût de déploiement.

Limites :

- **Masquage plutôt qu'élimination de la vulnérabilité :** la distillation défensive ne supprime pas les faiblesses du modèle. Elle tend principalement à rendre les gradients moins exploitables, créant ainsi une illusion de robustesse sans éliminer les vulnérabilités sous-jacentes.
- **Contournement par les attaques Carlini & Wagner :** les auteurs ont montré qu'en optimisant directement les logits plutôt que les probabilités Softmax, il était possible de contourner efficacement la protection offerte par la distillation défensive et de retrouver des taux de succès très élevés.
- **Contournement des transformations d'entrée par BPDA :** Athalye et al. ont introduit l'algorithme *Backward Pass Differentiable Approximation* (BPDA), qui permet d'approximer les gradients à travers des transformations non différentiables. Cette technique rend à nouveau possibles les attaques basées sur les gradients, telles que PGD, malgré la présence des mécanismes de prétraitement.

1.5.2 Techniques de préservation de la confidentialité

Les systèmes d'IA critiques manipulent des données hautement sensibles, nécessitant des garanties de confidentialité formelles. Trois approches principales émergent de la littérature.

1.5.2.1 Confidentialité différentielle

La confidentialité différentielle (Differential Privacy) [6] est une technique conçue pour assurer la protection des données sensibles présentes dans les données d'apprentissage d'un modèle d'intelligence artificielle. Cette méthode se base sur l'intégration d'un bruit contrôlé aux données ou aux paramètres du modèle pour éviter l'identification d'un individu spécifique dans le jeu de données.

Apports : Cette approche assure de fortes garanties mathématiques en matière de protection de la vie privée et sert à réduire les risques d'attaques par inférence ou de fuite d'informations sensibles au cours du processus d'apprentissage.

Limites : L'introduction de bruit pourrait conduire à une baisse des performances du modèle, surtout lorsque le niveau de confidentialité recherché est élevé. L'un des principaux défis de cette méthode réside dans le compromis entre confidentialité et précision constitue.

1.5.2.2 Apprentissage fédéré

L'apprentissage fédéré (Federated Learning [6, 17]) est une méthode décentralisée dans laquelle les données restent sur les appareils locaux des utilisateurs. Uniquement les paramètres ou les gradients calculés localement sont envoyés à un serveur central, responsable de regrouper les mises à jour du modèle global. L'algorithme FedAvg agrège les mises à jour locales :

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_k^{t+1} \quad (1.2)$$

où w_k^{t+1} représente les poids du modèle après entraînement local sur le client k .

Apports : Cette méthode réduit considérablement les risques associés à la centralisation des données, encourage la préservation de la confidentialité tout en permettant l'entraînement collaboratif de modèles en utilisant différentes sources de données.

Limites : Le modèle d'apprentissage fédéré est exposé à certaines attaques, comme l'empoisonnement des modèles locaux et la reconstruction de données à partir des gradients échangés. Son application peut aussi entraîner des coûts significatifs en communication.

1.5.2.3 Chiffrement homomorphe

Le chiffrement homomorphe est une méthode de cryptographie qui offre la possibilité d'effectuer des opérations directement sur des données chiffrées sans nécessiter leur déchiffrement préalable [14]. Appliqué aux systèmes d'IA, il permet de réaliser des opérations d'entraînement ou d'inférence tout en préservant la confidentialité des informations manipulées.

Apports : Cette approche garantit un niveau de sécurité exceptionnel, car les données demeurent cryptées tout au long du processus. Elle représente une option encourageante pour les applications traitant des informations sensibles dans des contextes peu fiables.

Limites : Son coût computationnel élevé constitue le principal obstacle à son adoption. Les calculs sur des données cryptées sont généralement bien plus lents que les calculs classiques, ce qui restreint encore son déploiement dans les systèmes d'IA qui demandent des performances en temps réel.

1.5.3 Frameworks et architectures de sécurité ML

Plusieurs architectures et frameworks globaux ont été suggérés pour assurer la sécurité de l'intégrité du cycle de vie des systèmes ML, en plus des mesures de protection spécifiques.

1.5.3.1 NIST AI Risk Management Framework (AI RMF)

En 2023, le NIST a publié le cadre de gestion des risques pour l'intelligence artificielle, connu sous le nom de NIST AI Risk Management Framework (AI RMF), qui est spécifiquement conçu pour les systèmes d'IA. Il propose une approche allant de la phase de conception de l'IA à l'étape d'exploitation, dans le but d'identifier, d'évaluer et de contrôler les risques relatifs à la sécurité, la fiabilité et la gouvernance des modèles [22].

Apports : Ce framework offre une approche structurée pour la gestion des risques et encourage l'inclusion de la sécurité dans les projets d'IA dès leur phase de conception. Il est actuellement considéré comme une norme établie pour guider l'élaboration de systèmes d'IA fiables et responsables.

Limites : L'approche reste principalement orientée gouvernance et gestion des risques. Elle fournit peu de mécanismes techniques concrets permettant de protéger les modèles contre les attaques adversariales, le vol de modèles ou les attaques sur les données d'entraînement.

1.5.3.2 Secure AI Framework (SAIF)

Proposé par Google en 2023, le Secure AI Framework (SAIF) applique des principes de cybersécurité classiques aux systèmes d'intelligence artificielle. Il s'appuie sur une stratégie de défense en profondeur destinée à garantir la sécurité des données, des modèles, de l'infrastructure et des procédures tout au long du cycle de vie de l'intelligence artificielle [16].

Apports : SAIF est l'un des premiers cadres spécifiquement consacrés à la sécurité de l'IA. Il inclut plusieurs aspects cruciaux tels que la protection des modèles, la sécurisation de la chaîne d'approvisionnement en IA, la gestion des accès et l'identification des menaces.

Limites : Le cadre demeure principalement théorique et ne propose pas de structure opérationnelle précise. Les moyens de mise en application ainsi que les procédés d'évaluation de leur efficacité sont faiblement développés.

1.5.3.3 Microsoft AI Security Architecture

Microsoft offre une architecture multicouche conçue pour assurer la sécurité des systèmes d'IA déployés dans son environnement cloud Azure. Cette stratégie fusionne la gestion des identités, la

protection des informations, la sécurisation des modèles et la surveillance continue des services IA [18].

Apports : L'architecture assure une sécurité totale sur l'environnement d'exécution grâce à l'incorporation de mécanismes de contrôle d'accès, de chiffrement, de gestion des secrets et d'identification des menaces.

Limites : Cette solution reste fortement dépendante de l'écosystème Azure et se fonde sur des technologies propriétaires. Son utilisation dans des contextes variés ou indépendants du cloud de Microsoft pourrait s'avérer très complexe.

1.5.4 Synthèse critique et positionnement de nos travaux

L'analyse des travaux actuels révèle que la question de la sécurité des systèmes d'intelligence artificielle est actuellement traitée sous plusieurs axes complémentaires. Les recherches concernant les défenses adversariales ont pour objectif principal de renforcer la robustesse des modèles face aux attaques, alors que les méthodes de conservation de la confidentialité ont pour but de sauvegarder les données sensibles utilisées pendant l'entraînement ou l'inférence. De même, divers frameworks comme le NIST AI RMF, le SAIF ou le Microsoft IA security Architecture, offrent des stratégies plus englobantes qui incluent la gestion des risques et la sécurité tout au long du cycle de vie des systèmes d'intelligence artificielle.

TABLE 1.2 – Synthèse des principales approches de sécurisation des systèmes d'IA

Approche	Domaine couvert	Principaux apports	Limites identifiées
Défenses adversariales	Robustesse des modèles	Amélioration de la robustesse face aux attaques.	Protection partielle et coût d'entraînement élevé.
Techniques de préservation de la confidentialité	Protection des données	Réduction des risques de divulgation des données.	Impact possible sur les performances et la précision.
NIST AI RMF	Gestion des risques IA	Cadre méthodologique structuré couvrant l'ensemble du cycle de vie des systèmes d'IA.	Orientée gouvernance avec peu de mécanismes techniques.
SAIF	Sécurisation globale des systèmes IA	Approche de défense en profondeur pour les Systèmes d'IA.	Framework essentiellement conceptuel.
Microsoft AI Security Architecture	Sécurité des plateformes IA	Protection multicouche des données, modèles et services IA intégrés au cloud.	Architecture fortement dépendante de l'écosystème Microsoft Azure.

Bien que ces travaux aient apporté une grande contribution, ils présentent néanmoins des limites.

Les mécanismes de défense examinés sont fréquemment ciblés sur une menace spécifique, alors que les frameworks existants restent généralement théoriques ou orientés vers des aspects spécifiques tels que la gouvernance, les processus ou la gestion des accès. Il existe peu de solutions qui offrent une architecture opérationnelle associant simultanément la protection des données, la robustesse des modèles, la sécurité applicative, la détection d'anomalies et la traçabilité des opérations.

Dans ce contexte, notre travail vise l'élaboration d'une Architecture de sécurité à 5 couches pour les systèmes d'IA, combinant ces divers mécanismes de protection dans une approche unifiée et aisément déployable en contexte opérationnel.

1.6 Éthique et cadre réglementaire des modèles IA

L'intégration croissante de l'intelligence artificielle dans des domaines sensibles soulève d'importants défis éthiques et réglementaires. L'utilisation des modèles d'IA nécessite l'application de principes de transparence, d'équité et de responsabilité pour prévenir les discriminations algorithmiques et les violations de la vie privée. De plus, plusieurs cadres réglementaires, tels que le **RGPD**, l'**EU AI Act** et la **norme ISO 42001**, ont été développés pour garantir une gouvernance responsable des systèmes d'IA. Cette section examine les principes fondamentaux de l'éthique en IA ainsi que les réglementations en vigueur.

1.6.1 Principes fondamentaux de l'éthique en IA

- **Transparence et explicabilité des modèles :**

Un aspect clé de l'éthique des modèles d'IA est la transparence. Les utilisateurs et les régulateurs doivent pouvoir comprendre comment un modèle prend ses décisions. D'après [19], l'absence de transparence peut engendrer un manque de confiance et des fautes aux implications sérieuses, en particulier dans des domaines tels que la santé ou le système judiciaire. Il est essentiel d'assurer l'interprétabilité des modèles en identifiant les caractéristiques ayant influencé une décision.

- **Justice algorithmique et réduction des biais**

L'équité algorithmique vise à garantir que les modèles IA prennent des décisions justes et non discriminatoires. Toutefois, les biais dans les données d'entraînement ou dans la conception des modèles peuvent conduire à des résultats inéquitables.

1.6.2 Régulations et normes

L'intégration de l'intelligence artificielle dans les domaines critiques nécessite un respect rigoureux des réglementations nationales et internationales. Ces régulations ont pour objectif de prévenir les abus, de sauvegarder les libertés individuelles et de réguler l'utilisation de technologies qui pourraient être intrusives ou discriminatoires.

- **RGPD (Règlement Général sur la Protection des Données) :**

Introduit en mai 2018 au sein de l'Union européenne, le **RGPD** (Règlement UE 2016/679) établit un cadre légal rigoureux pour la gestion des données personnelles, y compris les données biométriques. Il offre aux personnes le droit à la transparence, à la correction, à la suppression et à l'explication des décisions automatisées [9]. Dans le contexte sécuritaire, toute application de reconnaissance faciale ou d'analyse comportementale doit respecter ces obligations.

- **Loi n°2017-20 du Bénin sur la protection des données personnelles**

Le Bénin s'est également doté d'un cadre juridique national avec la **Loi 2017-20 du 20 avril 2018** [1], relative à la protection des données à caractère personnel. Cette loi crée l'Autorité de Protection des Données Personnelles (**APDP**), chargée de veiller à la conformité des traitements. Les systèmes IA déployés par les forces de sécurité doivent être déclarés à l'**APDP**, et les citoyens doivent être informés des traitements automatisés dont ils peuvent faire l'objet.

- **AI Act de l'Union européenne (proposé en 2021)**

L'**EU AI Act** (Règlement européen sur l'IA) [5] est le premier texte de loi à classer les systèmes d'IA par niveaux de risques. Les systèmes utilisés dans les domaines de la sécurité publique, comme la surveillance biométrique en temps réel ou la reconnaissance faciale, sont classés à **haut risque**, voire **interdits** dans certains cas (article 5). Ce texte impose des exigences fermes en matière de transparence, d'explicabilité, de robustesse, de contrôle humain et de gestion des biais. Il prévoit également la réalisation d'audits périodiques et la tenue d'une documentation exhaustive des systèmes.

- **Norme ISO/IEC 42001 sur les systèmes de management de l'IA (2023)**

Cette norme propose un guide pour concevoir, déployer et perfectionner un système de gestion des technologies d'IA au sein des organisations [4]. Elle contient des instructions concernant : la gestion des risques associés à l'IA, l'éthique, la clarté des décisions algorithmiques et la responsabilité des concepteurs et des utilisateurs. Dans le contexte de la sécurité publique, cette norme peut constituer un fondement pour une gouvernance stricte des systèmes d'IA mis en œuvre par l'État.

Conclusion

La diversité des vulnérabilités identifiées, conjuguée à la complexité croissante des attaques observées, montre qu'il ne suffit plus de concevoir des modèles performants. Il est désormais indispensable d'intégrer des mécanismes de protection tout au long du cycle de vie des modèles IA. Face à ces enjeux, il devient indispensable d'examiner de manière approfondie les méthodes de sécurisation, en prenant en compte aussi bien les contraintes techniques que les exigences éthiques spécifiques aux systèmes déployés dans les domaines sensibles.

Méthodologie et Architecture

Introduction

Ce chapitre présente l'expérimentation menée pour valider la méthode de protection des modèles d'intelligence artificielle suggérée dans ce mémoire. Nous étudions les faiblesses d'un modèle standard dans le contexte d'une étude de cas de classification d'images utilisée pour détecter des objets dangereux, puis nous mesurons l'efficacité des stratégies de défense mises en place. Cela inclut l'apprentissage adversarial et l'incorporation d'une stratégie DevSecOps dans le processus du cycle de vie du modèle.

2.1 Démarche méthodologique

La démarche méthodologique de cette recherche s'appuie sur une approche scrupuleuse dont l'objectif est d'examiner et d'implémenter des stratégies de sécurisation des systèmes IA déployés dans des environnements sensibles. Cette partie décrit les démarches entreprises pour atteindre les objectifs définis, offrant ainsi une vue d'ensemble du processus de recherche.

2.1.1 Collecte des Données

La collecte des données repose dans un premier temps sur l'identification des méthodes et outils pertinents pouvant intervenir dans la sécurisation des systèmes d'IA critiques et dans un second temps identification et l'étude d'un cas d'étude nous permettant d'illustrer l'architecture de sécurité que nous proposons.

2.1.2 Modélisation d'une Architecture de sécurité

Nous avons procédé à la modélisation d'une architecture de sécurité modulaire basée sur les principes de défense en profondeur, en tenant compte des particularités des systèmes d'apprentissage automatique. L'architecture suggérée se compose de multiples couches de protection complémentaires qui englobent tout le cycle de vie des systèmes d'IA : la sécurisation des données d'apprentissage, le renforcement du modèle par le biais d'un entraînement adversarial, et la protection lors de l'in-

férence avec la détection d'exemples adversariaux. Des éléments de monitoring et d'audit assurent une surveillance constante du comportement du modèle en production.

2.1.3 Mise en place d'un cas expérimental

Un cas expérimental illustrant l'architecture proposée a été mis en place afin de tester son opérationnalité. Il a exigé la création d'un environnement d'essai intégral et répliquable. Le processus expérimental inclut : un modèle de référence, un outil de création d'attaques adversariales (FGSM, C&W), ainsi que des systèmes d'évaluation automatisés pour mesurer la précision, la robustesse et le temps d'inférence. Le modèle a été déployé dans un environnement conteneurisé avec Docker, exposant une API REST pour simuler des conditions opérationnelles réalistes.

2.1.4 Simulation et Évaluation des Résultats

Des simulations ont été réalisées sur les scénarios d'attaque et de défense, conformément à un protocole d'évaluation rigoureux. L'évaluation s'articule autour d'une étude comparative afin de mettre en évidence l'avantage de notre approche de sécurisation par rapport aux méthodes traditionnelles et aux structures existantes. Nous avons comparé notre approche avec des modèles non protégés (baseline), des modèles qui ne s'appuient que sur des techniques de défense conventionnelles. Cette comparaison permet d'évaluer les améliorations en robustesse, d'estimer l'impact sur la performance et de situer notre apport par rapport aux meilleures pratiques du moment.

2.2 Méthode de sécurisation des systèmes IA dans les environnements critiques

Face aux menaces qui ont été présentées dans le chapitre précédent, nous proposons une architecture de sécurisation globale et modulaire pour les systèmes d'IA déployés dans les environnements critiques. Cette architecture s'inspire des principes de défense en profondeur de la cybersécurité traditionnelle, tout en les ajustant aux spécificités des systèmes d'apprentissage automatique. Notre approche couvre l'ensemble du cycle de vie des systèmes d'IA, depuis la collecte des données jusqu'à leur déploiement, en incluant la protection des données, la robustesse des modèles, la sécurité applicative, la surveillance opérationnelle et la traçabilité des actions .

2.2.1 Architecture globale de protection des systèmes IA

Ce travail propose une architecture qui cherche à incorporer de manière générale et continue la sécurité des systèmes d'intelligence artificielle tout au long de leur cycle de vie. Elle s'appuie sur une stratégie en plusieurs couches, associant des dispositifs de sécurité applicative, de protection du modèle et de sécurité de l'infrastructure. Chaque composant de l'architecture vise des objectifs de sécurité précis, depuis la protection des données et du modèle jusqu'à la supervision du déploiement et de l'utilisation en production.

2.2.1.1 Architecture en cinq couches de sécurité

Notre architecture est structurée en deux piliers complémentaires : un ensemble de quatre **couches opérationnelles** alignées verticalement, et une couche de gouvernance transverse. Les **couches opérationnelles** (Couche 1 à Couche 4) assurent la sécurisation séquentielle du cycle de vie du système IA, de la donnée brute au déploiement. Parallèlement, la **couche de gouvernance et de monitoring** (Couche 5) agit comme un plan de contrôle indépendant, garantissant la surveillance et la conformité de l'ensemble du pipeline. La figure 2.1 présente cette vue d'ensemble.

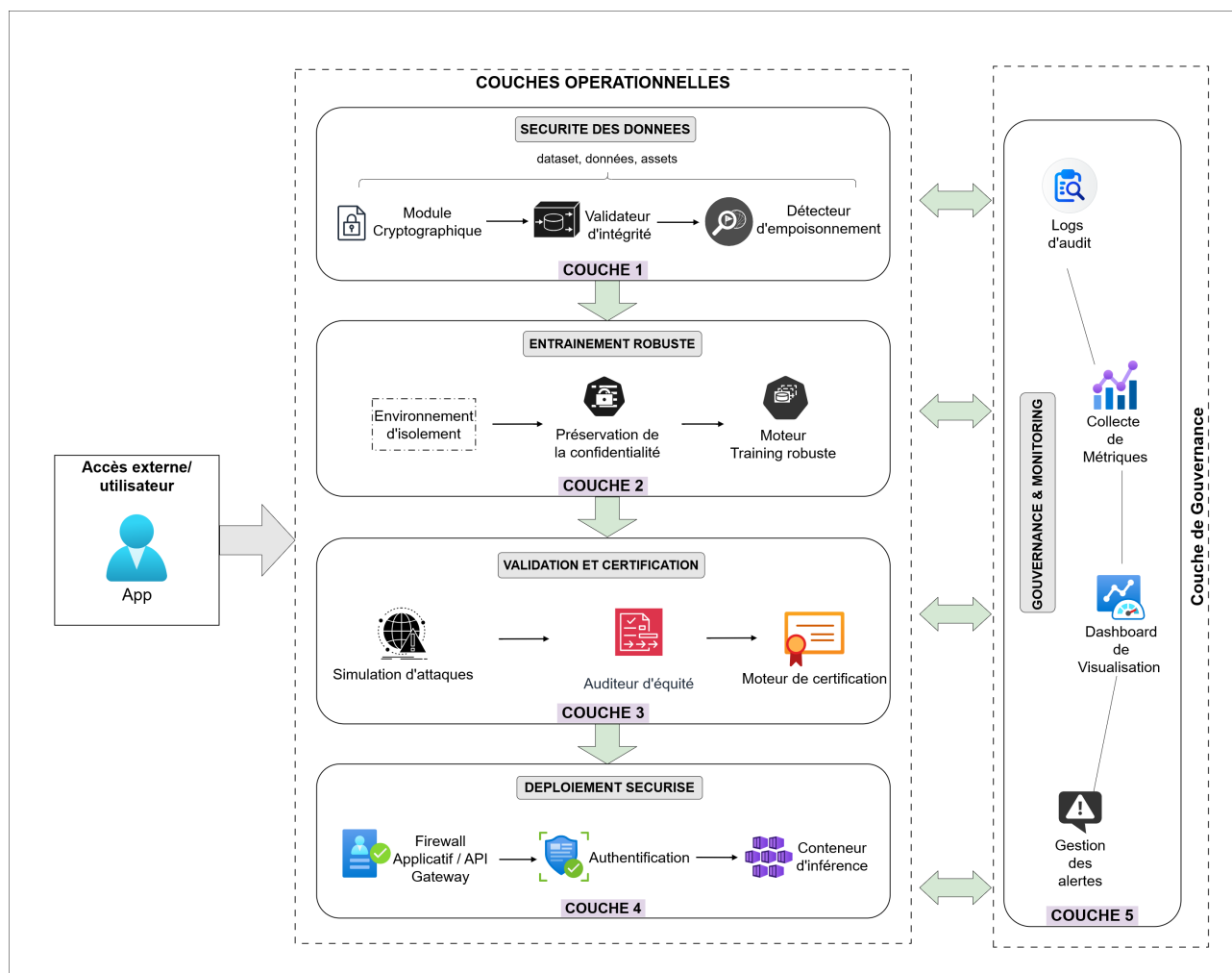


FIGURE 2.1 – Architecture globale de sécurité à 5 couches

Fonctionnement global de l'architecture :

L'architecture de sécurité proposée est structurée en **deux piliers complémentaires** : quatre couches opérationnelles séquentielles (Couche 1 à 4) assurant la sécurisation progressive du cycle de vie du système IA, et une couche de gouvernance transverse (Couche 5) agissant comme plan de contrôle indépendant. Les couches opérationnelles suivent un flux bloquant : la **Couche 1** assure l'intégrité et la qualité des données avant entraînement via des vérifications statistiques, une détection d'empoisonnement et un stockage chiffré. La transition vers la Couche 2 n'est autorisée qu'après validation complète. La **Couche 2** réalise un entraînement robuste et sécurisé en environnement isolé, intégrant l'adversarial training, la confidentialité différentielle et une signature cryptographique du modèle

produit. Le modèle entraîné franchit ensuite la **Couche 3** où il subit une série de tests adversariaux, d'équité et de conformité réglementaire dans un sandbox sécurisé. Seul un modèle satisfaisant tous les critères obtient son certificat de conformité pour accéder à la production. Enfin, la **Couche 4** déploie le modèle certifié dans un environnement hautement sécurisé, protégé par un WAF, une authentification JWT avec RBAC, un détecteur d'anomalies en temps réel et des logs d'audit immuables.

Parallèlement à ces quatre couches opérationnelles, la **Couche 5** assure une surveillance continue 24h/24 de l'ensemble du pipeline via un tableau de bord centralisé, un système de gestion des logs, et des alertes intelligentes.

Description des couches de sécurité :

1. Couche 1 : Protection et préparation des données (Data Security layer)

L'objectif de cette couche est d'assurer l'intégrité, la confidentialité et le niveau élevé de qualité des données avant leur intégration dans le processus d'apprentissage.

Composants clés :

- **Service de vérification des données** : Ce système intègre diverses étapes de vérification pour identifier les irrégularités, les informations altérées ou les tentatives de contamination.
- **Module de détection des empoisonnements** : Ce module, spécifiquement élaboré pour repérer les tentatives d'empoisonnement des données, analyse les données récentes en les comparant aux distributions passées.
- **Service d'anonymisation et de pseudonymisation** : Emploie des méthodes comme privacy-preserving afin de sauvegarder les informations personnelles.
- **Système de stockage crypté** : Les données sont conservées avec un cryptage lorsqu'elles sont au repos et avec TLS lors du transit.

Mécanismes de protection :

- Contrôle syntaxique et sémantique des informations entrantes
- Identification d'irrégularités statistiques avec seuils évolutifs
- Isolement automatique des données suspectes
- Gestion de version inaltérable avec hachage cryptographique (Exemple : SHA-256)
- Piste d'audit exhaustive de toutes les opérations sur les données.

L'intégrité des données agissant comme un prérequis, la transition vers la **Couche 2 (Entraînement)** ne s'opère qu'une fois la phase de validation de la Couche 1 finalisée et approuvée.

2. Couche 2 : Entraînement sécurisé (Training Security Layer)

Cette couche a pour objectif d'assurer un apprentissage robuste et sécurisé des modèles en intégrant des mesures de protection dès la phase d'apprentissage.

Composants clés :

- **Pipeline d'entraînement sécurisé** : Ce pipeline automatise l'apprentissage en incorporant des checkpoints réguliers et un système de récupération en cas d'erreur. Chaque étape du processus est encapsulée et mise en œuvre dans un environnement séparé.
- **Module d'entraînement Adversarial** : Intègre de manière automatique des cas adversariaux au cours du processus d'entraînement.
- **Service de Confidentialité Différentielle** : Introduit un bruit gaussien ajusté aux gradients durant l'apprentissage afin d'assurer la confidentialité différentielle.
- **Versionnage du registre de modèles** : Conserve les modèles entraînés avec l'ensemble de leurs métadonnées détaillées (hyperparamètres, indicateurs de performance, paramètres de formation). Chaque modèle est authentifié numériquement à l'aide d'une signature numérique pour assurer son intégrité.

Mécanismes de protection :

- Environnement d'apprentissage isolé
- Suivi en direct des indicateurs d'apprentissage pour repérer les irrégularités
- Sauvegarde incrémentielle des points de contrôle avec contrôle d'intégrité
- Identification des backdoors par l'examen des paramètres du modèle
- Validation croisée systématique afin d'éviter le surapprentissage

Une fois l'entraînement finalisé sous les mesures de protection de cette couche, le modèle franchit la deuxième porte de contrôle pour accéder à la **Couche 3**, où sa résistance réelle sera mise à l'épreuve par des simulations d'attaques.

3. Couche 3 : Validation et certification (Validation Security Layer)

L'objectif ici est de vérifier en détail la solidité, l'équité et le respect des normes des modèles avant leur mise en production.

Composants clés :

- **Série de tests adversariaux** : Ensemble complet de tests comprenant des attaques adversariales (comme : FGSM, PGD, C&W, DeepFool). Les tests sont réalisés dans un environnement sécurisé en sandbox pour prévenir toute contamination.
- **Auditeur pour les biais et l'équité** : Examine les performances du modèle à travers différentes catégories démographiques. Effectue des calculs de mesures d'équité et produit des rapports détaillés.
- **Évaluateur de robustesse** : Examine la résilience du modèle face aux perturbations naturelles (comme le bruit, le flou, les variations d'éclairage) et adversariales. Attribue une note globale de robustesse en se basant sur plusieurs critères avec des poids variés.
- **Service de validation de conformité** : Contrôle la conformité avec les normes réglementaires (comme RGPD, Loi sur l'IA, ISO 42001). Produit des certificats de conformité accompagnés de preuves cryptographiques (blockchain ou horodatage certifié).

Processus de validation se fait en plusieurs phases :

- (a) **Tests fonctionnels** : Contrôle des performances standards sur des ensembles de données de référence.
- (b) **Tests de sécurité** : Appréciation de la robustesse face aux attaques adversariales
- (c) **Tests d'équité** : Examen des préjugés et discriminations possibles
- (d) **Tests de confidentialité** : Contrôle pour s'assurer qu'il n'y a pas de fuite d'information
- (e) **Contrôles de conformité** : Vérification des exigences réglementaires et éthiques

Conditions de validation :

- *Taux de précision minimal* : 85% sur l'ensemble de test
- *Résilience face aux attaques* : plus de 60% de survie lors des attaques standard
- *Biais maximal autorisé* : écart de 5% entre les groupes
- *Score en matière de confidentialité* : conformité à la règle -differential privacy
- *Conformité totale* : toutes les exigences obligatoires sont respectées

Une fois la solidité du modèle validée par les simulateurs d'attaques, celui-ci obtient son certificat de conformité, lui permettant de quitter l'environnement de test pour être déployé en milieu opérationnel au sein de la **Couche 4**.

4. Couche 4 : Production sécurisée (Production Security Layer)

Cette couche de sécurité a pour objectif de mettre en œuvre et faire fonctionner les modèles validés dans un environnement de production très sécurisé et étroitement surveillé.

Composants clés :

- *API Gateway sécurisée* : Point d'entrée unique pour toutes les requêtes. Ici il faut mettre en œuvre l'authentification, le contrôle d'accès et la limitation du taux de requêtes (Ex : 100 requêtes par minute par défaut, modifiable par client).
- *Service de rate limiting et DDoS protection* : Met en œuvre des algorithmes de "token bucket" et de "sliding window" pour restreindre les demandes. On peut faire l'intégration avec des services tels que Cloudflare ou AWS Shield en vue d'une protection contre les attaques par déni de service distribué.
- *Module de détection d'anomalies en temps réel* : Examine les schémas de requêtes pour identifier les comportements suspects.
- *Service de réponse sécurisé* : Examine et approuve les réponses avant de les transmettre. Met en œuvre des méthodes de perturbation des outputs pour améliorer la confidentialité différentielle.

La mise en production devra avoir une **architecture de haute disponibilité** :

- Déploiement multi-instances avec load balancing
- Failover automatique avec health checks toutes les 5 secondes
- Circuit breaker pattern pour isoler les services défaillants
- Cache distribué (Redis) pour optimiser les performances

- Blue-green deployment pour les mises à jour sans interruption

En production, des **mécanismes de protection** suivants seront mis en œuvre :

- WAF (Web Application Firewall) pour filtrer les requêtes malveillantes
- Input sanitization et validation stricte
- Détection et blocage automatique des attaques adversariales
- Monitoring des dérives de distribution
- Rollback automatique en cas de dégradation des performances

Le déploiement effectif en Couche 4 marque l'entrée du modèle dans sa phase de vie active, dont le contrôle, l'audit et la conformité sont centralisés et régis par la **Couche 5**.

5. Couche 5 : Gouvernance et surveillance (Governance Layer)

L'objectif ici est de veiller à une surveillance constante, de respecter la réglementation et de gérer les incidents à travers toute l'architecture.

Composants clés :

- **Tableau de bord de monitoring centralisé** : Une interface unifiée (Exemple : Grafana) présentant le statut global du système, les mesures de performance, les alertes actives et les évolutions antérieures ; Personnalisable selon le rôle de l'utilisateur.
- **Système de gestion des logs** : Collecte, l'indexation et l'analyse des logs. Conservation minimale de 90 jours avec archivage à long terme. Des logs signés et horodatés pour assurer leur intégrité.
- **Service d'alerte intelligent** : Utilise des règles complexes et le machine learning pour détecter les anomalies et générer des alertes pertinentes. Intégration avec Slack ou email pour l'envoi des alertes.
- **Module de reporting de conformité** : Générez de manière automatique des rapports de conformité (RGPD, loi sur l'IA) avec l'ensemble des justificatifs nécessaires.
- **Centre de réponse aux incidents (IRT)** : Procédures automatisées pour gérer les incidents de sécurité. Guides préétablis pour les situations courantes.

Processus de gouvernance :

1. **Surveillance continue** : Monitoring 24/7 avec alertes automatiques
2. **Revue périodiques** : Audits mensuels de sécurité et trimestriels de conformité
3. **Gestion des changements** : Processus d'approbation pour toute modification
4. **Formation continue** : Sessions régulières pour les équipes sur les menaces émergentes
5. **Amélioration continue** : Incorporation des feedbacks et des leçons tirées de l'expérience.

Ainsi, la Couche 5 garantit la cohérence et la résilience du système en surveillant les quatre couches opérationnelles antérieures. Une fois la hiérarchie mise en place, il est important d'analyser comment l'approche DevSecOps automatise et fluidifie les échanges entre ces divers niveaux de sécurité.

2.2.2 Approche DevSecOps dans la protection des systèmes IA.

Dans ce travail, l'adoption de la méthode DevSecOps est justifiée par le besoin d'intégrer en permanence la sécurité dans le cycle de vie des systèmes d'intelligence artificielle. À la différence des systèmes logiciels traditionnels, les systèmes d'IA sont confrontés à des risques spécifiques associés aux données, à l'entraînement et à l'inférence. Par conséquent, une sécurité exclusivement mise en œuvre lors de la phase de déploiement n'est pas suffisante. Ainsi, le DevSecOps facilite l'automatisation des vérifications de sécurité, renforce la traçabilité des modèles et garantit une supervision continue des vulnérabilités, tout en préservant la flexibilité des processus de développement et de déploiement.

A travers notre architecture ressort les aspects DevSecOps suivants :

- **Un Pipeline CI/CD sécurisé en 05 étapes** : la validation des données ; le scan de sécurité (vulnérabilités de code, dépendances, Scan des secrets) ; l'entraînement sécurisé (environnement isolé, adversarial training, monitoring des métriques) ; les tests adversariaux (attack, biais et équité) et le déploiement sécurisé (déploiement progressif, monitoring, Rollback).
- **Les "Security Gates" automatisés** : Qui sont des points de contrôle automatique qui bloquent la progression du pipeline si certains critères ne sont pas respectés. Ils agissent comme des gardes de sécurité automatiques qui vérifient que tout est en ordre avant de laisser passer.
- **La Gestion automatique des versions et rollback** : Si une nouvelle version du modèle pose problème (baisse de performance, vulnérabilité découverte), il faut pouvoir revenir rapidement à une version stable. Le rollback automatique surveille l'évolution des métriques en production ; en cas de détection de problème, on revient automatiquement à la version précédente.
- **Containérisation avec Docker** : pour la reproductibilité, l'isolement, la portabilité et la scalabilité.
- **Monitoring et alerting en production** : identifier rapidement des problèmes de chute de performance du modèle, d'attaques adversariales, de biais émergents ou de fuites de données afin de prendre des mesures avant qu'ils n'affectent les utilisateurs.

2.2.3 Recommandations éthiques des systèmes d'IA

Il est impératif de concevoir et d'utiliser les systèmes d'intelligence artificielle en conformité avec des principes éthiques largement admis, tels que :

- **Transparence et explicabilité** : Les systèmes d'IA devraient offrir une explication claire des critères de prise de décision. Dans le contexte de la sécurité, cela implique qu'un citoyen faussement identifié par un système de reconnaissance faciale doit être en mesure de comprendre les fondements algorithmiques sur lesquels repose cette identification.
- **Justice et équité algorithmique** : Il est impératif que l'IA ne reproduise ni n'amplifie les discriminations sociétales existantes. Cependant, diverses recherches ont prouvé que certains algorithmes de reconnaissance faciale présentent une précision nettement inférieure pour les visages féminins ou les personnes à la peau sombre. Dans le contexte des contextes critiques, cette disparité suscite particulièrement l'inquiétude.

- **Responsabilité** : Il faut pouvoir identifier et tenir pour responsables les concepteurs et utilisateurs des systèmes d'IA. On ne peut pas attribuer l'erreur uniquement à la machine. Le concept de responsabilité humaine demeure essentiel dans les décisions cruciales, surtout en ce qui concerne les arrestations, la surveillance ou les interventions de police.
- **Respect de la vie privée** : L'acquisition ou le traitement de données biométriques ou comportementales doivent être réalisés de manière proportionnelle, justifiée et en accord avec les normes de protection des données personnelles.

Plusieurs organisations internationales ([IEEE](#), [UNESCO](#), [OCDE](#)) ont adopté et consolidé ces principes, qui sont désormais un élément essentiel des meilleures pratiques requises pour le développement d'IA critiques.

2.3 Outils et Technologies

L'implémentation de notre architecture de sécurisation nécessite un ensemble d'outils et de technologies soigneusement sélectionnés pour leur maturité, leur performance et leur compatibilité avec les contraintes opérationnelles des systèmes IA en milieu critique. Cette section présente la stack technologique retenue et justifie nos choix à la lumière des exigences de sécurité, de performance et de reproductibilité.

2.3.1 Environnement de développement

Python : Python est le langage prédominant utilisé pour l'implémentation des modèles et des pipelines de sécurité. Ce choix est basé sur la richesse des bibliothèques de machine learning et de sécurité présentes dans l'univers Python. L'écosystème Python dédié à l'apprentissage automatique, incluant des bibliothèques comme NumPy, scikit-learn et PyTorch, favorise l'incorporation de techniques de sécurisation avancées tout en conservant une base de code aisée à gérer.

PyTorch : On a opté pour PyTorch comme framework d'apprentissage profond à cause de sa souplesse et de son approche dynamique pour la création de graphes computationnels [26]. Cette caractéristique permet l'usage de techniques d'entraînement adversaire et procure une maîtrise fine du processus d'apprentissage, essentielle pour l'incorporation de stratégies défensives personnalisées. Ce choix a aussi été influencé par la communauté active de PyTorch et la disponibilité d'implémentations standards pour les méthodes de résistance aux attaques adversariales.

Google Colab / Jupyter Notebook : Jupyter, un environnement de développement interactif, facilite l'expérimentation par itérations et la documentation du code. En outre, Google Colab offre un accès à des ressources GPU gratuites, ce qui simplifie l'entraînement de modèles sans la nécessité d'une infrastructure locale coûteuse. Cette approche favorise la duplication des expériences et facilite le partage des résultats au sein de la communauté scientifique.

2.3.2 Bibliothèques de sécurité et de robustesse

Adversarial Robustness Toolbox (ART) : La bibliothèque de référence en matière de robustesse adversariale, l'Adversarial Robustness Toolbox, a été conçue par IBM Research [23]. ART propose

des versions normalisées et soigneusement documentées pour :

- La génération d'attaques adversariales (FGSM, PGD, C&W, DeepFool, etc.)
- Les méthodes de défense (adversarial training, feature squeezing, defensive distillation)
- Les métriques de robustesse permettant d'évaluer quantitativement la résistance des modèles

Foolbox : Foolbox est un framework Python autonome, ne dépendant d'aucune bibliothèque de deep learning, qui facilite l'évaluation de la robustesse des modèles face à diverses attaques adversariales. Sa compatibilité intrinsèque avec PyTorch et sa facilité d'utilisation font de cet outil un atout précieux pour l'analyse comparative de la robustesse. Foolbox se distingue par sa capacité à générer des perturbations adversariales de petite envergure, fournissant ainsi des aperçus sur la distance réelle aux frontières de prise de décision.

2.3.3 Infrastructure de déploiement

Docker : Docker simplifie l'intégration de toutes les dépendances et configurations, réduisant ainsi les risques liés aux différences entre les environnements de développement et de production. Les conteneurs fournissent également une sécurité en minimisant les impacts potentiels d'une possible brèche de sécurité.

FastAPI : FastAPI est un framework moderne pour la création d'API REST efficaces en Python. Sa vérification automatique des données d'entrée, fondée sur les type hints Python et Pydantic, assure une protection contre les entrées mal structurées. FastAPI, qui prend en charge de manière native les opérations asynchrones, offre des performances optimales tout en garantissant une faible exposition aux vulnérabilités grâce à sa conception sécurisée par défaut.

Nginx : Nginx assure le rôle de reverse proxy, offrant plusieurs fonctionnalités de sécurité indispensables :

- Limitation du taux de requêtes pour éviter les attaques par déni de service .
- Load balancing pour répartir la charge entre plusieurs instances du modèle
- Terminaison TLS/SSL pour chiffrer les communications
- Filtrage des requêtes malformées avant qu'elles n'atteignent l'application

2.3.4 Monitoring et observabilité

Prometheus + Grafana : Prometheus et Grafana constituent un ensemble d'outils de surveillance qui facilite la collecte et l'affichage en temps réel des indicateurs de performance et de sécurité.

ELK Stack (Elasticsearch, Logstash, Kibana) : Le stack ELK est une solution unifiée pour la collecte, le traitement et l'affichage des logs. Elasticsearch se charge de la sauvegarde et de la recherche efficace des informations, Logstash agrège et harmonise les journaux en provenance de diverses sources, alors que Kibana offre la possibilité de les examiner et de les représenter graphiquement à travers des tableaux de bord.

2.3.5 Justification des choix technologiques

L'option de cette stack technologique est basée sur divers paramètres complémentaires : maturité et la robustesse des outils, leur capacité à être compatibles et interopérables grâce à des normes standards, leur performance, la faisabilité d'accès par le biais de solutions open-source, l'évolutivité de l'architecture conteneurisée, et enfin leur utilisation dans le milieu de la recherche scientifique qui assure une comparaison avec les meilleures pratiques actuelles. Cette pile de technologies offre donc une base robuste qui combine rigueur scientifique, efficacité opérationnelle et pragmatisme économique.

Conclusion

Ce chapitre a détaillé la méthode et l'infrastructure technique qui appuient notre approche pour sécuriser les systèmes d'IA déployés dans des environnements critiques. Notre démarche rigoureuse, axée sur l'acquisition de données, la modélisation, l'expérimentation et l'évaluation, garantit la crédibilité scientifique de nos travaux. Par la suite, le chapitre détaille l'architecture précise de notre solution de sécurisation, clarifiant les mesures de protection instaurées à chaque étape et leur intégration dans un système opérationnel et performant. Le choix de la stack technologique, combinant des outils éprouvés du monde Python avec des bibliothèques spécialisées dans la résilience face aux adversaires, fournit la base essentielle pour l'implémentation pratique de notre système de protection.

Implémentation de l'approche proposée

Introduction

Ce chapitre illustre la mise en œuvre pratique de notre technique de sécurisation au moyen d'une expérimentation exhaustive avec le modèle MobileNetV2, dans le cadre d'un système de test pour la détection d'objets dangereux pour les forces de l'ordre. Le but est de prouver la viabilité et l'efficacité de l'architecture en cinq couches proposée au Chapitre 2.

3.1 Présentation du cas expérimental

3.1.1 Objectif et scénario de l'expérimentation

L'objectif principal de cette expérimentation est de mettre en œuvre deux approches de développement de systèmes de détection d'objets dangereux par vision par ordinateur :

1. **Approche Baseline (vulnérable)** : Un modèle MobileNetV2 entraîné de manière standard via transfer learning, représentant les pratiques courantes de développement IA sans considération particulière pour la robustesse adversariale qui permettra de faire une évaluation initiale de la robustesse du modèle face aux attaques adversariales.
2. **Approche Secured (robuste)** : Un modèle MobileNetV2 identique point de vue structure, mais entraîné avec des défenses qui intègrent l'entraînement adversarial et des mécanismes de détection d'anomalies.

Le scénario de cette expérimentation est la **détection binaire d'armes à feu**, un cas dont l'utilisation critique dans les systèmes de sécurité où une attaque adversariale pourrait permettre de faire passer un objet dangereux pour un objet sûr. Le problème est présenté sous la forme d'une classification binaire : **classe "safe"** (objets sûrs) versus **classe "dangerous"** (armes à feu). Cette simplification aide à focaliser l'examen sur la robustesse adversariale plutôt que sur la complexité taxonomique, tout en facilitant l'interprétation des résultats de sécurité.

L'analyse de ces deux approches facilitera la quantification de l'amélioration de robustesse apportée par les mécanismes de défense, d'évaluer le trade-off entre performance sur données propres

et robustesse adversariale, et de valider la faisabilité pratique de déploiement de modèles robustes en production.

3.1.2 Présentation du dataset

Le dataset utilisé comprend 1 896 images réparties de manière égale entre deux classes : dangerous (images d'armes à feu provenant de Kaggle) et safe (images issues du jeu de données COCO) évitant ainsi le déséquilibre de données. Le tableau 3.1 illustre la distribution des données en suivant le protocole de répartition est de 70% pour l'entraînement, 15% pour la validation et 15% pour le test, ce qui assure une distribution équilibrée et une évaluation précise du modèle, tout en fournissant une base appropriée pour l'apprentissage adversarial.

TABLE 3.1 – Statistiques du dataset

Ensemble	Classe "safe"	Classe "dangerous"	Total	Ratio
Entraînement	699	699	1 398	73.7%
Validation	147	147	294	15.5%
Test	102	102	204	10.8%
Total	948	948	1 896	100%

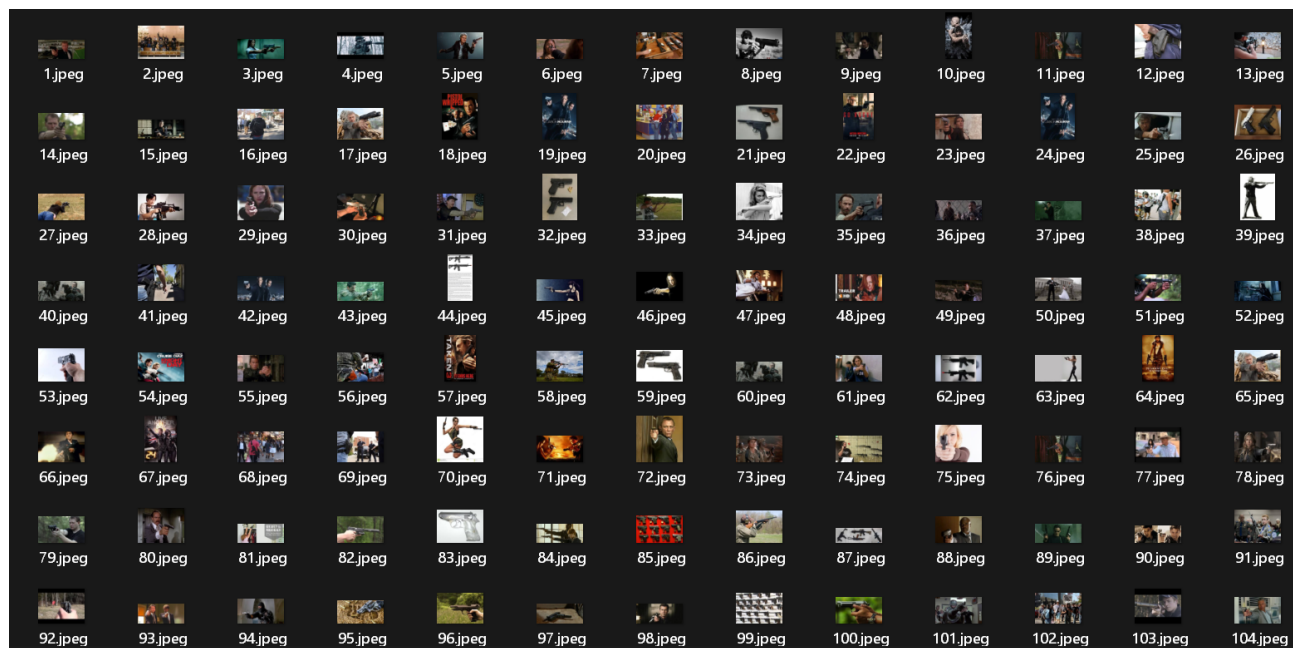


FIGURE 3.1 – Exemples de la classe "dangerous"

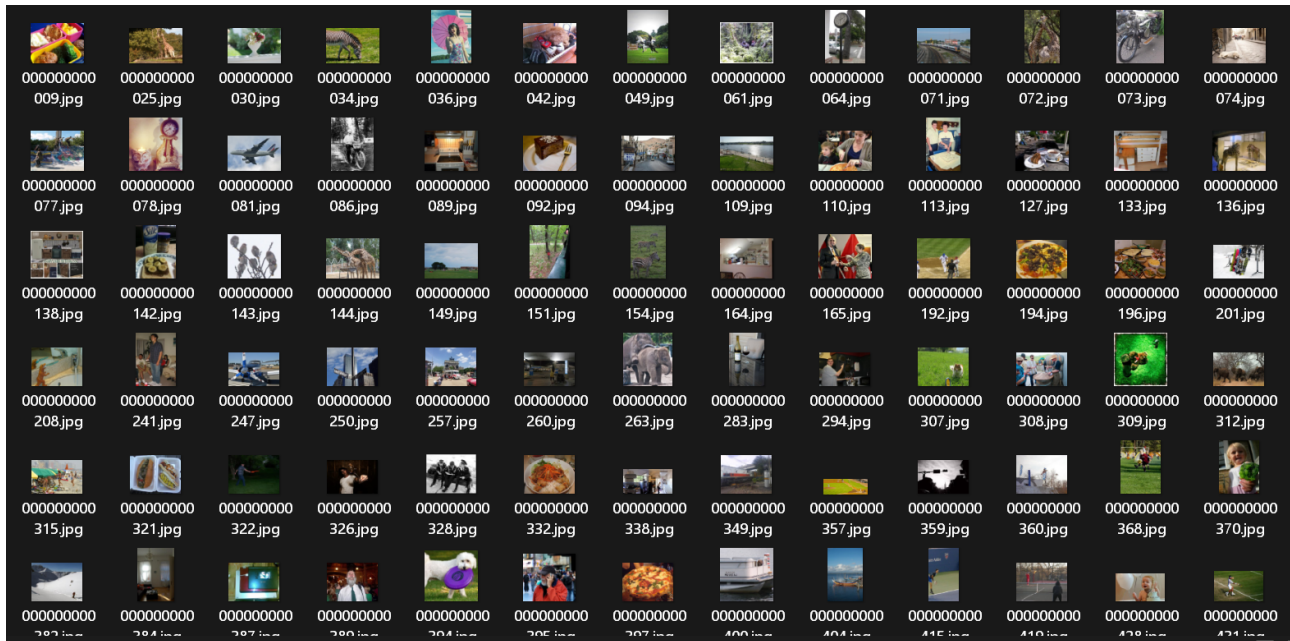


FIGURE 3.2 – Exemples de la classe “safe”

3.2 Choix du modèle de base

Pour cette expérimentation, nous notre choix s’est porté sur **MobileNetV2** [27] comme architecture de base pour notre système de détection. MobileNetV2 est un réseau de neurones convolutifs léger conçu par Google, qui se distingue par une architecture reposant sur les *inverted residual blocks* et les *linear bottlenecks*. Ces innovations architecturales offrent de bonnes performances (72% sur ImageNet) pour un coût computationnel faible (3,5 M de paramètres contre 25,6 M pour ResNet50).

Ce choix est basé sur trois avantages principaux : son efficacité computationnelle, qui permet une mise en œuvre sans GPU ; sa configuration modulaire, adaptée à l’entraînement adversarial ; et son support du transfer learning, qui simplifie l’adaptation des poids préentraînés d’ImageNet pour la détection d’armes à feu.

3.3 Développement et entraînement sécurisé

Compte tenue des vulnérabilités identifiées sur le modèle baseline, il est indispensable de mettre en place un processus d’entraînement qui intègre des mécanismes de défense adversariale. Dans cette section, nous présentons l’environnement de développement Docker, le pipeline d’apprentissage sécurisé avec adversarial training FGSM, et la protection cryptographique mis en œuvre afin d’améliorer la robustesse du modèle.

3.3.1 Environnement de développement Docker

L’environnement de développement est basé sur Docker Compose pour garantir la reproductibilité, l’isolation des services, et la portabilité entre divers environnements (local, cloud, CI/CD).

Description des 8 services

Un ensemble complet de 8 services interconnectés est déployé dans l'environnement de développement assurant le développement, les tests, le monitoring et la visualisation du système de détection.

1. **dev** : entraînement du modèle et mise en place de l'API.
2. **test** : analyse et tests de sécurité et de robustesse.
3. **postgres** : base de données pour utilisateurs, logs et métriques.
4. **redis** : cache et gestion des sessions [JWT](#), files asynchrones et limites de requêtes.
5. **prometheus** : collecte des métriques et gestion des alertes.
6. **grafana** : Dashboard de visualisation pour suivre les performances et les paramètres de sécurité.
7. **node-exporter** : surveillance des ressources système (CPU, RAM, réseau).
8. **web** : interface utilisateur facilitant des tests de prédictions avec les modèles.

3.3.2 Pipeline d'entraînement sécurisé

Le pipeline d'entraînement sécurisé intègre trois étapes critiques : la sécurisation des données , l'adversarial training avec FGSM , et la sauvegarde chiffrée avec signature cryptographique. La figure [3.3](#) illustre le flux complet.

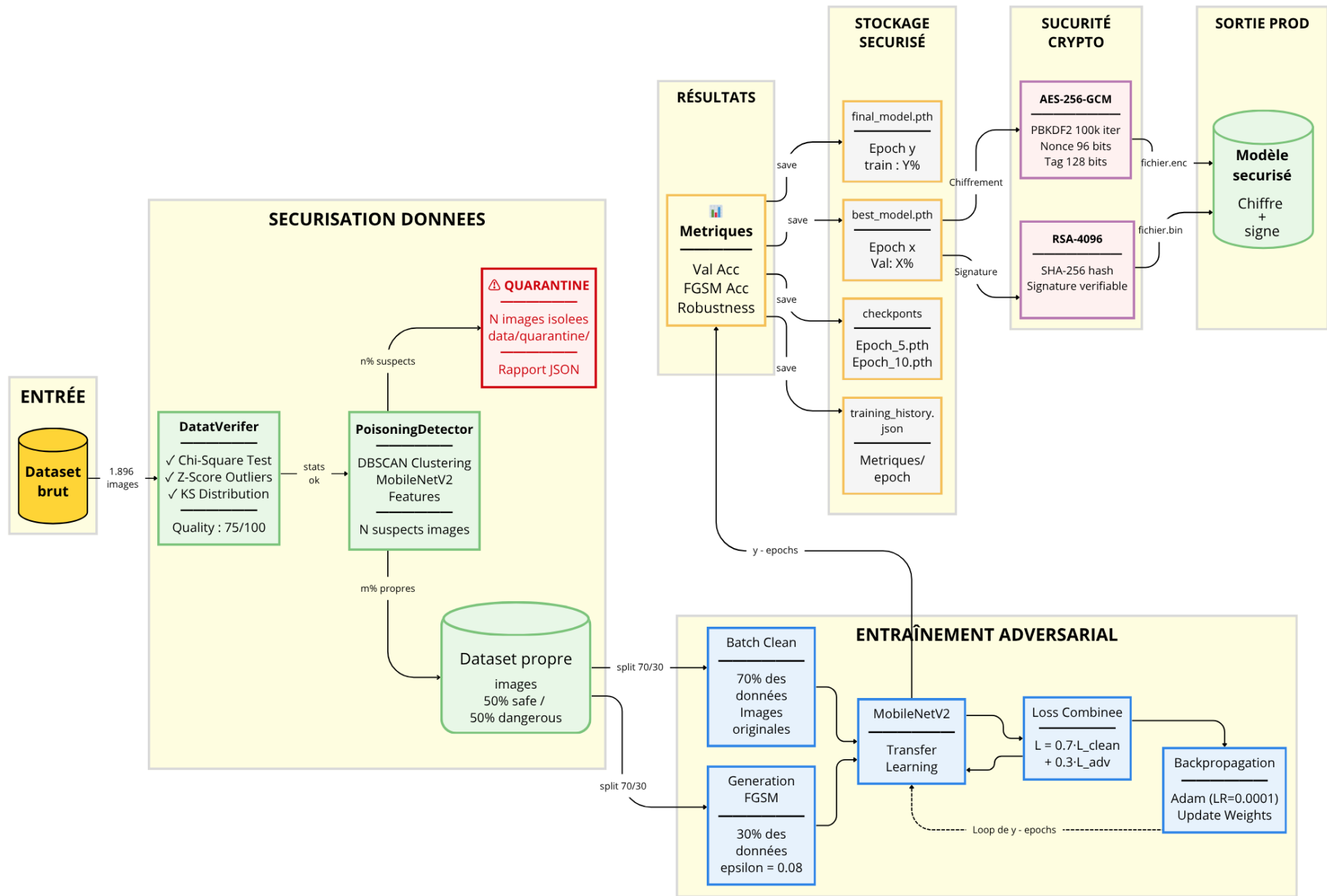


FIGURE 3.3 – Pipeline d'entraînement sécurisé

3.3.2.1 Sécurisation des données pré-entraînement

L'objectif est de garantir la qualité et l'intégrité du dataset avant entraînement en détectant les anomalies statistiques et le data poisoning. Avant tout entraînement, le dataset subit deux vérifications automatiques :

1. Validation statistique (`DataVerifier`) : Applique trois tests statistiques sur le dataset :

- *Test Chi-Square* : Vérifie l'équilibre des classes (948 safe, 948 dangerous) avec p -value seuil 0.05
- *Test Z-Score* : Détecte les outliers pixels avec seuil $|z| > 3.0$ (distribution normale des intensités)
- *Test Kolmogorov-Smirnov* : Compare les distributions train/test pour détecter les drifts

```
1 verifier = DataVerifier(train_data_path)
2 report = verifier.verify_data()
3
4 chi2_stat, p_value = scipy.stats.chisquare([948, 948])
5 z_scores = (pixel_values - mean) / std
6 outliers = np.where(np.abs(z_scores) > 3.0)[0]
7 ks_stat, ks_p = scipy.stats.ks_2samp(train_dist, test_dist)
```

Listing 3.1 – Validation statistique (version courte)

2. Détection de poisoning (`PoisoningDetector`) : Utilise DBSCAN sur les features MobileNetV2 pour identifier les échantillons suspects. Les images avec features anormales (outliers DBSCAN, label = -1) sont mises en quarantaine.

```
1 detector = PoisoningDetector()
2 poisoning_report = detector.detect_poisoning(dataset_path=train_data_path)
3
4 feature_extractor = mobilenet_v2(pretrained=True).features
5 features = [feature_extractor(img).flatten() for img in dataset]
6
7 clustering = DBSCAN(eps='auto', min_samples='auto').fit(features)
8 suspicious_indices = np.where(clustering.labels_ == -1)[0]
```

Listing 3.2 – Détection de poisoning (version courte)

Les images détectées sont automatiquement déplacées vers `data/quarantine/` avec un rapport JSON traçable. Le dataset d'entraînement est réduit aux images propres équilibrées, garantissant l'absence de contamination.

3.3.2.2 Adversarial Training avec FGSM

Ici nous visons à renforcer la robustesse du modèle face aux attaques adversariales en l'entraînant simultanément sur des images normales et des images perturbées.

Principe de l'adversarial training :

Il est question d'augmenter le dataset d'entraînement avec des exemples adversariaux produits dynamiquement. À l'inverse de l'entraînement traditionnel qui se base uniquement sur des données propres, l'adversarial training oblige le modèle à apprendre une classification correcte tant sur les images normales (non perturbées) que sur les images adversariales (avec perturbations FGSM)

Cette stratégie permet au modèle de construire des représentations internes plus solides, moins vulnérables aux perturbations adversariales mineures. Ainsi, le modèle est formé pour se focaliser sur des attributs visuels essentiels plutôt que sur des motifs superficiels que pourrait exploiter un attaquant.

Fonctionnement durant l'entraînement :

Lors de chaque batch d'entraînement, le processus comporte trois phases :

1. *Forward pass sur données propres* : Évaluation du loss sur les images originales
2. *Génération d'exemples FGSM* : Création d'images adversariales en intégrant des perturbations calculées à l'aide du gradient de la loss ($\epsilon = 0.08$)
3. *Forward pass sur données adversariales* : Évaluation de la loss sur les images altérées
4. *Optimisation combinée* : Actualisation des poids dans le but de minimiser conjointement les deux losses

La loss totale est une combinaison pondérée : $\mathcal{L}_{total} = 0.7 \times \mathcal{L}_{clean} + 0.3 \times \mathcal{L}_{adversarial}$. Cette pondération facilite la préservation d'une bonne précision sur les données propres tout en renforçant de manière significative la robustesse face aux attaques adversariales.

```

1 def _train_epoch_adversarial(self, epoch):
2     for data, target in self.train_loader:
3         output_clean = self.model(data)
4         clean_loss = self.criterion(output_clean, target)
5
6         data.requires_grad = True
7         loss_for_grad = self.criterion(self.model(data), target)
8         loss_for_grad.backward()
9
10        fgsm_data = data + 0.08 * data.grad.sign()
11        fgsm_data = torch.clamp(fgsm_data, 0, 1).detach()
12
13        output_adv = self.model(fgsm_data)
14        adv_loss = self.criterion(output_adv, target)
15
16        total_loss = 0.7 * clean_loss + 0.3 * adv_loss
17        total_loss.backward()
18        torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_norm=1.0)
19        self.optimizer.step()

```

Listing 3.3 – Adversarial Training FGSM (version courte)

3.3.2.3 Configuration d'entraînement

Les hyperparamètres utilisés pour l'entraînement sécurisé sont définis dans `src/experiments/secured/train`

```

1 config = {
2     'epochs': 30, //
3     'batch_size': 32,

```

```

4     'learning_rate': 0.0001,
5     'adversarial_epsilon': 0.08, // Magnitude de perturbation FGSM
6     'clean_ratio': 0.7, // Pondération 70/30 des losses
7     'fgsm_ratio': 0.3,
8     'dropout': 0.45, // Taux de dropout élevé pour régularisation
9     'gradient_clipping': True, // Activation du clipping de gradients (norm
    =1.0)
10    'early_stopping_patience': 8,
11    'differential_privacy': False
12 }

```

Listing 3.4 – Configuration d'entraînement sécurisé

3.3.2.4 Sauvegarde des checkpoints d'entraînement

Durant l'entraînement, plusieurs artefacts sont sauvegardés automatiquement dans `models/secured/` :

```

1 torch.save({
2     'model_state_dict': self.model.state_dict(),
3     'optimizer_state_dict': self.optimizer.state_dict(),
4     'epoch': 7,
5     'val_acc': 89.46,
6     'config': config
7 }, 'models/secured/best_secured_model.pth')
8
9 with open('models/secured/training_history.json', 'w') as f:
10     json.dump(history, f, indent=2)

```

Listing 3.5 – Sauvegarde des modèles (version courte)

Quatre artefacts sont générés :

1. `best_secured_model.pth` : Meilleur modèle (epoch 7, val_acc 89.46%)
2. `final_secured_model.pth` : Modèle final (epoch 11)
3. `secured_model_epoch_*.pth` : Checkpoints périodiques (reprise d'entraînement)
4. `training_history.json` : Historique complet (courbes d'apprentissage)

3.3.2.5 Chiffrement AES-256-GCM du modèle

Afin de protéger ses poids contre l'extraction non autorisée et le reverse engineering, le meilleur modèle est chiffré avec AES-256-GCM (Galois/Counter Mode) avant déploiement en production. Ce mode de chiffrement authentifié garantit à la fois la confidentialité (chiffrement) et l'intégrité (tag d'authentification 128 bits). Le fichier chiffré peut être déployé en production sans risque d'extraction des poids par un attaquant.

```

1 storage = EncryptedStorage(password="SecureAI_2024_Production")
2
3 metadata = storage.encrypt_pytorch_model(

```

```

4     'models/secured/best_secured_model.pth',
5     'models/secured/encrypted/best_secured_model_encrypted.enc'
6 )

```

Listing 3.6 – Chiffrement AES-256-GCM (version courte)

3.3.2.6 Signature RSA-4096 pour non-répudiation

Le modèle est signé avec RSA-4096 (padding PSS) pour garantir qu'il n'a pas été modifié ou remplacé par un modèle malveillant garantissant son authenticité et son intégrité. La clé privée est maintenue sur le serveur d'entraînement, la clé publique peut être déployée en production pour vérification. En production, bien avant son chargement, l'API vérifie automatiquement la signature du modèle, rejetant tout fichier modifié ou non signé.

```

1 private_key = rsa.generate_private_key(public_exponent=65537, key_size=4096)
2 public_key = private_key.public_key()
3
4 model_hash = hashlib.sha256(model_data).digest()
5 signature = private_key.sign(
6     model_hash,
7     padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.
8         MAX_LENGTH),
9     hashes.SHA256())
10
11 with open('models/secured/best_secured_model_signature.bin', 'wb') as f:
12     f.write(signature)

```

Listing 3.7 – Signature RSA-4096 (version courte)

3.3.3 API de développement et interface web

L'API de développement (`src/api/app.py`) expose les 02 modèles entraînés via FastAPI pour tests interactifs durant le développement.

3.3.3.1 Chargement sécurisé du modèle au démarrage

Au démarrage de l'API, le modèle sécurisé subit un processus de vérification en trois étapes : (1) vérification de la signature RSA-4096, (2) déchiffrement AES-256-GCM, (3) chargement PyTorch. Si la signature échoue, l'API refuse de démarrer.

```

1 @app.on_event("startup")
2 async def load_models():
3     with open('models/secured/best_secured_model_signature.bin', 'rb') as f:
4         signature = f.read()
5     public_key = serialization.load_pem_public_key(open('...').read())
6     model_hash = hashlib.sha256(open('...').read()).digest()
7

```

```
8     public_key.verify(signature, model_hash, padding.PSS(...), hashes.SHA256
    ())
9
10    storage = EncryptedStorage(password=os.getenv('MODEL_PASSWORD'))
11    decrypted_path = storage.decrypt_pytorch_model('...enc', '/tmp/model.pth'
    )
12
13    checkpoint = torch.load(decrypted_path)
14    secured_model.load_state_dict(checkpoint['model_state_dict'])
```

Listing 3.8 – Chargement sécurisé (version courte)

Le processus en trois étapes garantit qu'aucun modèle corrompu ne peut être chargé. Si la signature échoue, l'API refuse de démarrer et lève une exception critique.

3.3.3.2 Endpoints de l'API de développement

L'API expose plusieurs groupes d'endpoints de pour le développement et la validation.

- **Endpoints de prédiction** : appels pour le modèle baseline ou secured (/predict/baseline, /predict/secured) et les métriques de service (/metrics)
- **Endpoints d'authentification et sécurité** : gestion de l'authentification [JWT](#), du [WAF](#) (/security/waf/status) et anomalies détectées (/security/anomalies/recent)
- **Endpoints d'audit** : consultation et export de logs et statistiques (/audit/logs, /audit/stats, /audit/export)

3.3.3.3 Interface web de développement

L'interface web servie par Nginx est organisée en plusieurs pages HTML statiques : une page d'accueil pour juxtaposer les prédictions baseline et secured, une page de connexion JWT, un tableau de bord admin pour la gestion et l'audit, ainsi qu'un tableau de bord de monitoring en temps réel.

3.4 Architecture et déploiement en production

Le modèle sécurisé entraîné en développement doit être déployé dans un environnement de production garantissant la disponibilité, la sécurité des accès, et la traçabilité complète des opérations. Cette partie détaille l'architecture de production Docker qui comprend deux zones de sécurité essentielles : protection des accès et détection d'anomalies (Couche 4) puis la gouvernance, audit et surveillance (Couche 5).

3.4.1 Architecture Docker de production

L'architecture de production est construite autour d'une infrastructure conteneurisée orchestrée par Docker Compose (docker-compose.prod.yml), utilisant Nginx comme proxy inverse principal et deux instances d'API FastAPI qui alimentent les modèles baseline et secured.

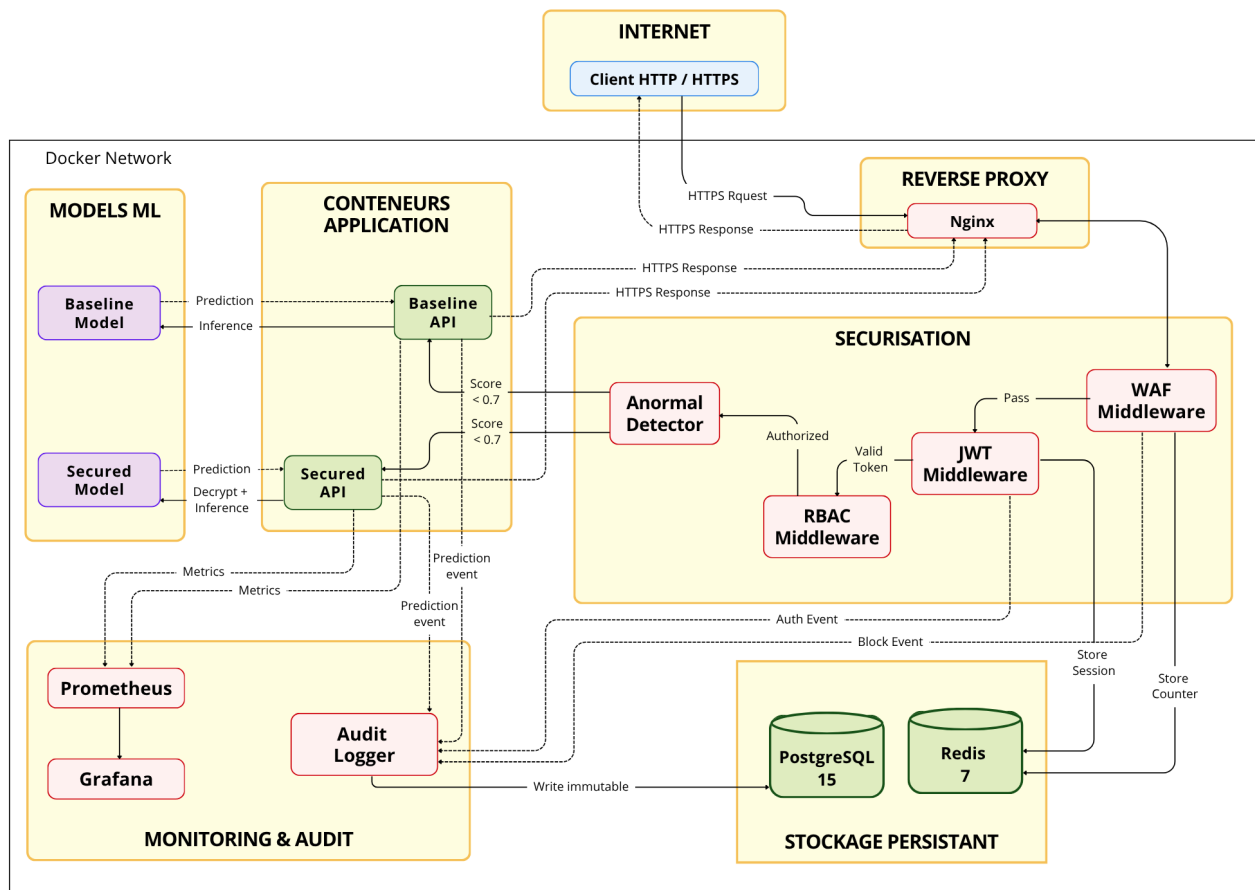


FIGURE 3.4 – Architecture de production : flux utilisateur, zones de sécurité, et composants

Description du flux : Avant d’atteindre les **modèles d’IA**, chaque requête client passe à travers une série de couches de sécurité. Le **WAF** filtre les attaques connues et restreint le trafic, puis procède à l’authentification **JWT** et à la gestion d’accès basée sur les rôles. L’outil de **détection d’anomalies** examine le comportement inhabituel avant de diriger la requête vers l’**API** correspondante (baseline ou sécurisée). Les modèles réalisent l’**inférence**, pendant que chaque opération est de façon immuable enregistrée dans **PostgreSQL** et surveillée en direct via **Prometheus/Grafana**. **Redis** garantit la conservation des sessions et des compteurs de limitation afin d’assurer des performances optimales.

Stack Docker de production :

Huit services sont déployés dans un réseau isolé dans l’environnement de production.

- **2 API FastAPI** : baseline-api et secured-api dédiées aux modèles respectifs
- **Nginx** : Reverse proxy HTTP/HTTPS avec équilibrage de charge et terminaison SSL
- **PostgreSQL 15** : Base de données pour utilisateurs, logs d’audit, et IPs bloquées par le **WAF**
- **Redis 7** : Cache pour sessions **JWT**, compteurs de rate limiting, et prédictions fréquentes
- **Prometheus** : Collecte de métriques en temps réel (requêtes/sec, latence, accuracy)
- **Grafana** : Tableaux de bord de surveillance avec des alertes paramétrées
- **Web Frontend** : Interface HTML/JS fournie par Nginx pour soumission d’images

3.4.2 Sécurisation en production

La sécurité en production implémente quatre couches de défense protégeant l'API contre les accès non autorisés, les abus, et les comportements anormaux.

Web Application Firewall (WAF) Le **WAF** (`src/security/waf.py`) applique un rate limiting strict par IP et détecte les patterns d'attaque :

- **Rate Limiting** : 100 requêtes/minute par IP (compteur Redis). Dépassement ==> HTTP 429, IP bloquée 15 minutes
- **Détection SQL Injection** : Regex détectant `UNION SELECT, DROP TABLE, -. Match ==> HTTP 403, IP blacklistée`
- **Détection XSS** : Regex détectant `<script>, onerror=, javascript:.. Match ==> requête rejetée`

Les IPs bloquées sont stockées dans PostgreSQL avec timestamp, raison et durée du blocage.

Authentification **JWT** (JSON Web Token) et contrôle d'accès **RBAC** (Role-Based Access Control)

L'authentification **JWT** (`src/security/auth.py`) utilise l'algorithme HS256. Le système définit trois rôles avec permissions distinctes :

```

1 class UserRole(Enum):
2     ADMIN = "admin"          # Tous droits
3     AGENT = "agent"          # Predictions + audit
4     GUEST = "guest"          # Predictions uniquement
5
6 class Permission(Enum):
7     PREDICT = "predict"
8     VIEW_AUDIT = "view_audit"
9     EXPORT_AUDIT = "export_audit"
10    MANAGE_USERS = "manage_users"
11    VIEW_METRICS = "view_metrics"
12 }
```

Listing 3.9 – Définition des rôles et permissions

Le flux d'authentification génère deux tokens : un access token (validité 15 minutes) contenant `user_id`, `role` et `permissions`, et un refresh token (validité 7 jours) stocké dans Redis.

Détection d'anomalies comportementales Le détecteur d'anomalies (`src/security/anomaly_detector.py`) analyse le comportement de chaque IP en temps réel sur plusieurs dimensions :

```

1 class AnomalyType(Enum):
2     SUSPICIOUS_PATTERN = "suspicious_pattern"
3     BURST_ACTIVITY = "burst_activity"
4     REPEATED_FAILURES = "repeated_failures"
5     UNUSUAL_TIMING = "unusual_timing"
6     CONFIDENCE_DROP = "confidence_drop"
```

```
7 MODEL_SWITCHING = "model_switching"
```

Listing 3.10 – Types d'anomalies et niveaux de risque

Parmi les caractéristiques examinées, on retrouve la fréquence des requêtes (intervalle d'une minute), la variété des endpoints, le poids moyen des images, le rapport erreurs/succès, ainsi que l'intervalle de temps entre chaque requête. Un score d'anomalie est calculé, variant entre 0 et 1, et des actions sont déclenchées en fonction du seuil :

- **Score 0.7-0.85** : Supervision accrue, log des événements.
- **Score 0.85-0.95** : Restriction temporaire (50 req/min)
- **Score >0.95** : Blocage automatique 1 heure, notification admin

3.4.3 Gouvernance et audit

L'objectif est de garantir la traçabilité complète des opérations et l'observabilité temps réel du système.

Logs d'audit immuables L'audit logger (`src/monitoring/audit_logger.py`) enregistre chaque événement critique dans PostgreSQL avec garantie d'immuabilité via hash chain SHA-256. Les types d'événements tracés sont :

```
1 class EventType(Enum):
2     PREDICTION = "prediction"
3     ATTACK_DETECTED = "attack_detected"
4     VALIDATION_FAILED = "validation_failed"
5     API_ACCESS = "api_access"
6     MODEL_LOADED = "model_loaded"
7     SYSTEM_ERROR = "system_error"
8     SECURITY_CHECK = "security_check"
9     SECURITY_ALERT = "security_alert"
```

Listing 3.11 – Types d'événements d'audit

Chaque log contient :

- **PREDICTION** : Timestamp, IP source (pseudonymisée), user_id, model_type, prediction, confiance, durée d'inférence, hash de l'image
- **ATTACK_DETECTED** : IP, type d'attaque, payload, action prise
- **SECURITY_ALERT** : Score d'anomalie, features, niveau de risque

Chaque log contient le hash SHA-256 du log précédent, créant une blockchain d'audit. Les IPs sont pseudonymisées avec HMAC-SHA256 (pour la conformité [RGPD](#)).

Monitoring temps réel avec Prometheus et Grafana Prometheus (`configs/monitoring/prometheus.yml`) assure l'extraction des métriques des API toutes les 15 secondes.

Grafana affiche 4 dashboards pré-configurés :

1. **Overview** : Requêtes/sec, latence , taux d'erreur, uptime
2. **Security** : [WAF](#) blocks, anomalies, tentatives d'auth échouées, top IPs suspects
3. **Models** : Accuracy baseline vs secured, distribution prédictions, temps d'inférence
4. **Audit** : Logs générés, top users, hash chain integrity

Les règles d'alerte (`configs/monitoring/alert_rules.yml`) se déclenchent sur des seuils critiques :

- Taux d'erreur HTTP 5xx > 10% pendant 5 minutes ==> `HighErrorRate`
- Latence P95 > 2 secondes pendant 10 minutes ==> `HighLatency`
- Accuracy < 80% sur fenêtre 1h ==> `ModelDegradation`
- Score d'anomalie moyen > 0.8 pendant 15 minutes ==> `AnomalySpike`

Conclusion

Ce chapitre nous a présenté une implémentation expérimentale de l'architecture de sécurité proposé dans le chapitre précédent à travers un système de détection binaire d'arme à feu. Il illustre la mise en œuvre pratique des différentes mesures de sécurité pour chaque Couche identifiée dans l'architecture. Dans le chapitre suivant , nous présenterons les résultats obtenus lors cet processus d'expérimentation.

Résultats et discussion

Introduction

Ce chapitre présente les résultats expérimentaux obtenus lors de l'implémentation et l'évaluation de notre système de détection d'images malveillantes sécurisé. Nous présentons d'abord l'environnement de déploiement Docker, puis les résultats d'entraînement du modèle sécurisé, suivis de l'évaluation de la robustesse adversariale, de la présentation des interfaces et de l'API, et enfin une discussion comparative avec l'état de l'art et les limites identifiées.

4.1 Résultats obtenus

4.1.1 Résultats d'entraînement du modèle sécurisé

- Convergence et métriques d'entraînement

Le modèle sécurisé a été entraîné pendant 11 epochs avec adversarial training FGSM. La figure 4.1 présente l'évolution des métriques d'entraînement et de validation.

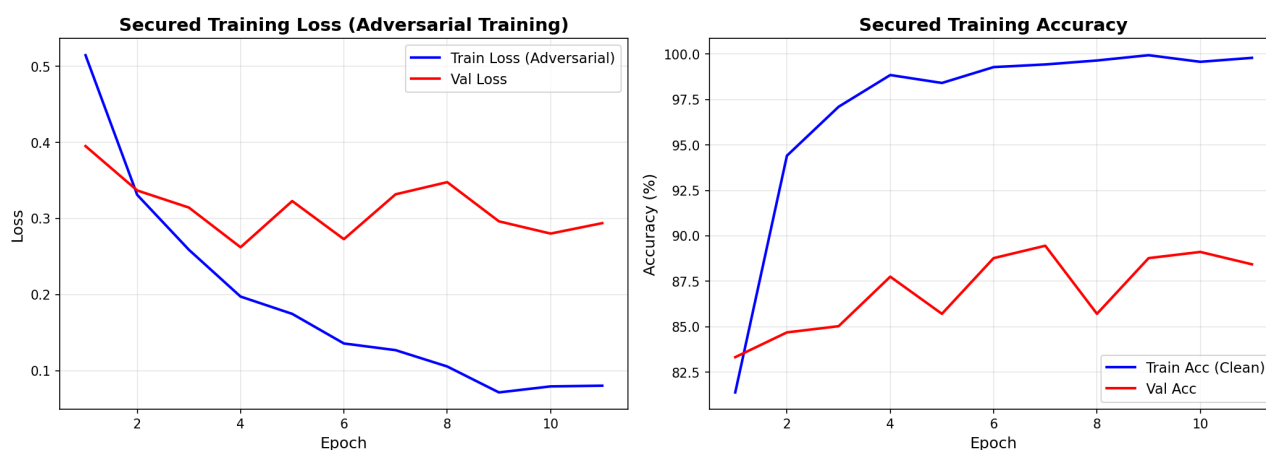


FIGURE 4.1 – Courbes d'entraînement du modèle sécurisé : loss et accuracy

Le meilleur modèle a été obtenu à l'époch 7 avec une validation accuracy de 89.46%. L'entraînement a été arrêté à l'époch 11 après détection de légers signes d'overfitting. Les résultats quantitatifs finaux sont présentés dans le tableau 4.1.

TABLE 4.1 – Résultats d'entraînement du modèle sécurisé

Métrique	Meilleur (Epoch 7)	Final (Epoch 11)
Train Accuracy	99.42%	99.78%
Train Loss	0.1268	0.0801
Val Accuracy	89.46%	88.44%
Val Loss	0.3319	0.2939

Les résultats présentés dans le 4.1 montrent que le modèle sécurisé conserve un niveau de précision globalement comparable à celui du modèle baseline, avec une légère variation liée à l'intégration des mécanismes de sécurité, notamment l'adversarial training et les techniques de régularisation. Cette stabilité des performances confirme que les mesures de sécurisation proposées n'entraînent pas une dégradation significative de la capacité de généralisation du modèle sur des données légitimes.

- **Comparaison avec le modèle baseline**

Le tableau 4.2 compare les performances du modèle baseline (entraînement classique) avec le modèle sécurisé (adversarial training).

TABLE 4.2 – Comparaison baseline vs sécurisé sur données propres

Métrique	Baseline	Sécurisé
Meilleure Val Accuracy	97.62% (Epoch 27)	89.46% (Epoch 7)
Val Loss finale	0.225	0.294
Nombre d'épochs	33	11
Overfitting	Modéré	Faible
Trade-off	-8.16% accuracy pour robustesse	

On observe une chute importante de la précision du modèle baseline lorsqu'il est soumis aux perturbations adversariales, traduisant sa forte vulnérabilité face à ce type d'attaques. À l'inverse, le modèle sécurisé présente une résistance nettement améliorée, avec une précision significativement plus élevée sous attaque. Le modèle sécurisé sacrifie donc 8.16% d'accuracy sur données propres

en échange d’une robustesse adversariale significativement améliorée. Cette différence s’explique par l’intégration de l’adversarial training lors de la phase d’apprentissage, qui permet au modèle de développer des représentations plus robustes face aux perturbations intentionnelles.

- **Évaluation de la robustesse adversariale**

Le modèle sécurisé a été testé contre des attaques FGSM ($\epsilon = 0.1$) sur un ensemble de 204 images de test. Les résultats sont présentés dans le tableau 4.3.

TABLE 4.3 – Robustesse FGSM : Baseline vs Sécurisé

Métrique	Baseline	Sécurisé	Amélioration
Clean Accuracy	97.55%	93.14%	-4.41%
Adversarial Accuracy	50.00%	68.14%	+18.14%
Attack Success Rate	50.00%	31.86%	-18.14%
Robustness Gap	47.50%	25.00%	-22.50%

Le tableau 4.3 montre que le modèle sécurisé améliore l’adversarial accuracy de 18.14 points (amélioration relative de +36.3%) et réduit le robustness gap de moitié (47.5% → 25.0%). Ces résultats montrent que, bien que le modèle sécurisé puisse présenter un léger compromis sur certaines métriques en conditions normales, il offre un gain substantiel en termes de robustesse et de fiabilité globale.

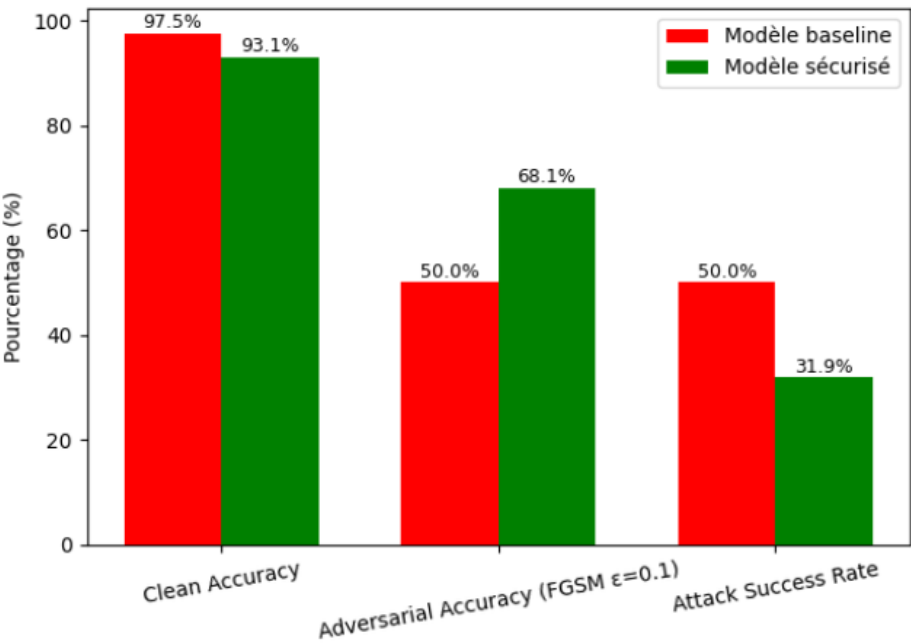


FIGURE 4.2 – Comparaison de robustesse FGSM : Baseline vs Sécurisé

Cette figure met en évidence la comparaison directe entre le modèle baseline et le modèle sécurisé, montrant une amélioration significative de la robustesse adversariale au prix d'une légère diminution des performances sur données propres.

4.1.2 Interfaces utilisateur et API

- Page d'authentification JWT

La page de login implémente l'authentification JWT avec trois rôles : admin, agent, guest. La figure 4.3 montre l'interface de connexion.

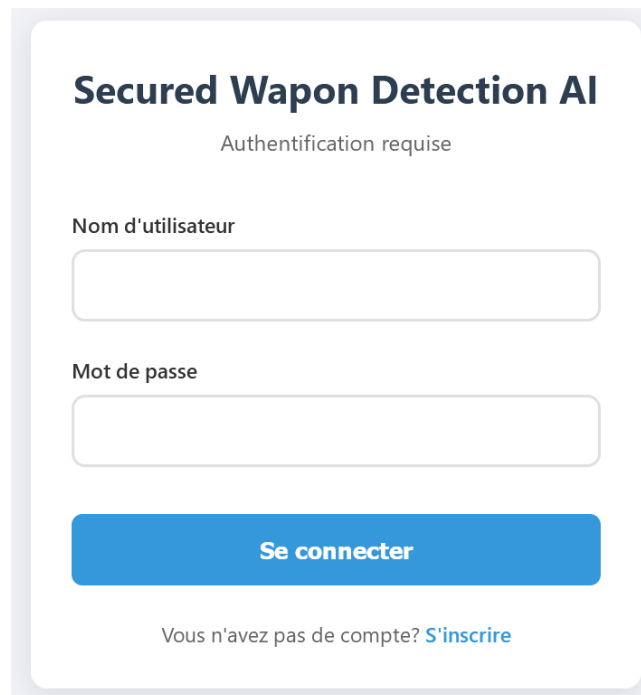


FIGURE 4.3 – Page de connexion avec authentification JWT

- Interface web de prédiction

L'interface web principale permet de soumettre une image et de faire une classification dangereux ou sûr, selon qu'une arme à feu soit détectée ou pas. On a la possibilité de faire le choix entre les deux modèles pour la prédiction. La figure 4.4 montre l'interface en action.

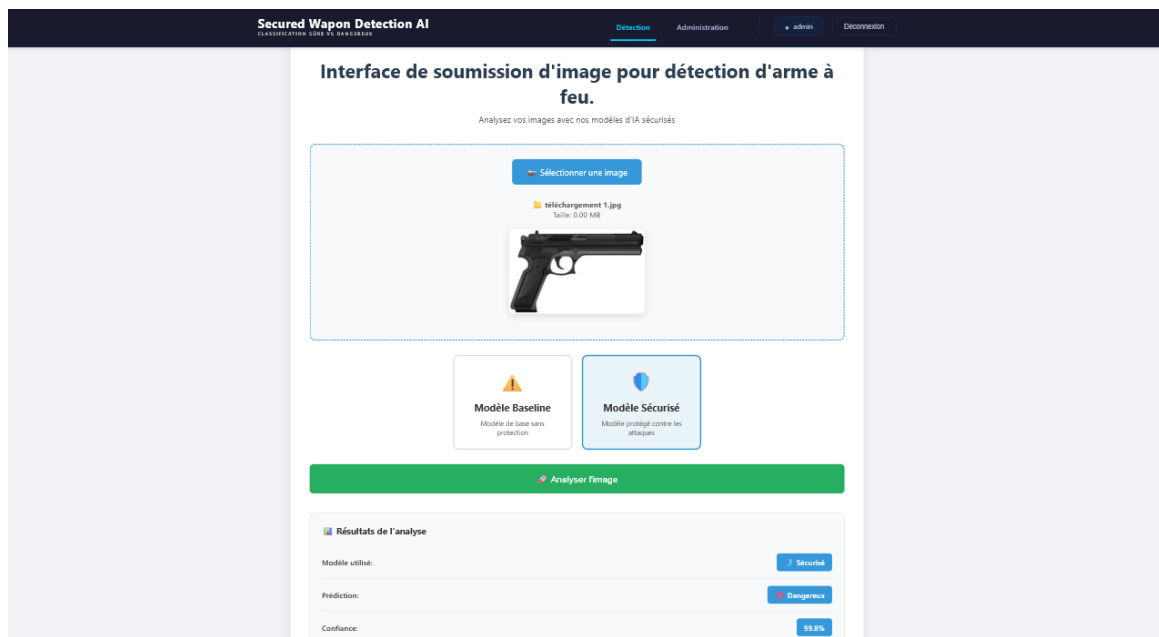


FIGURE 4.4 – Interface web de prédiction : classification Sûr / Dangereux

- **Interface web administrative**

Le dashboard administratif offre une vue complète de la sécurité, des utilisateurs, des logs d’audit, et des statistiques système. La figure 4.5 présente les fonctionnalités disponibles.

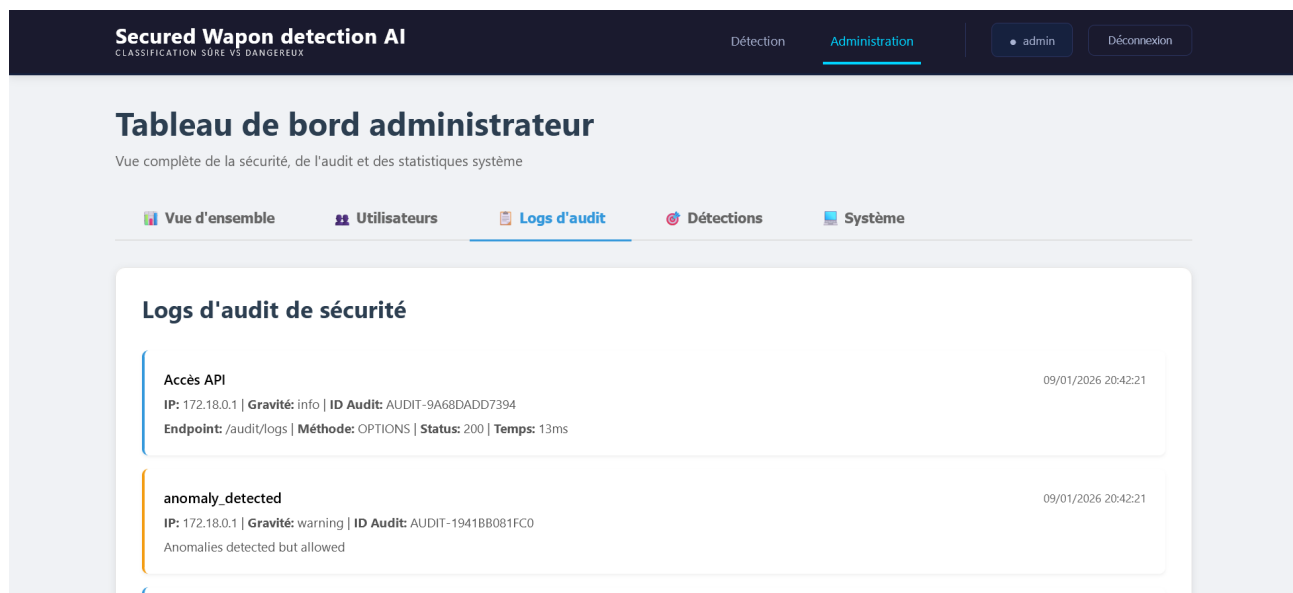


FIGURE 4.5 – Dashboard administratif : gestion utilisateurs et audit

- **Documentation API interactive (Swagger)**

FastAPI génère automatiquement une documentation Swagger UI interactive montrant les endpoints exposés par l’API. La figure 4.6 montre l’interface Swagger.

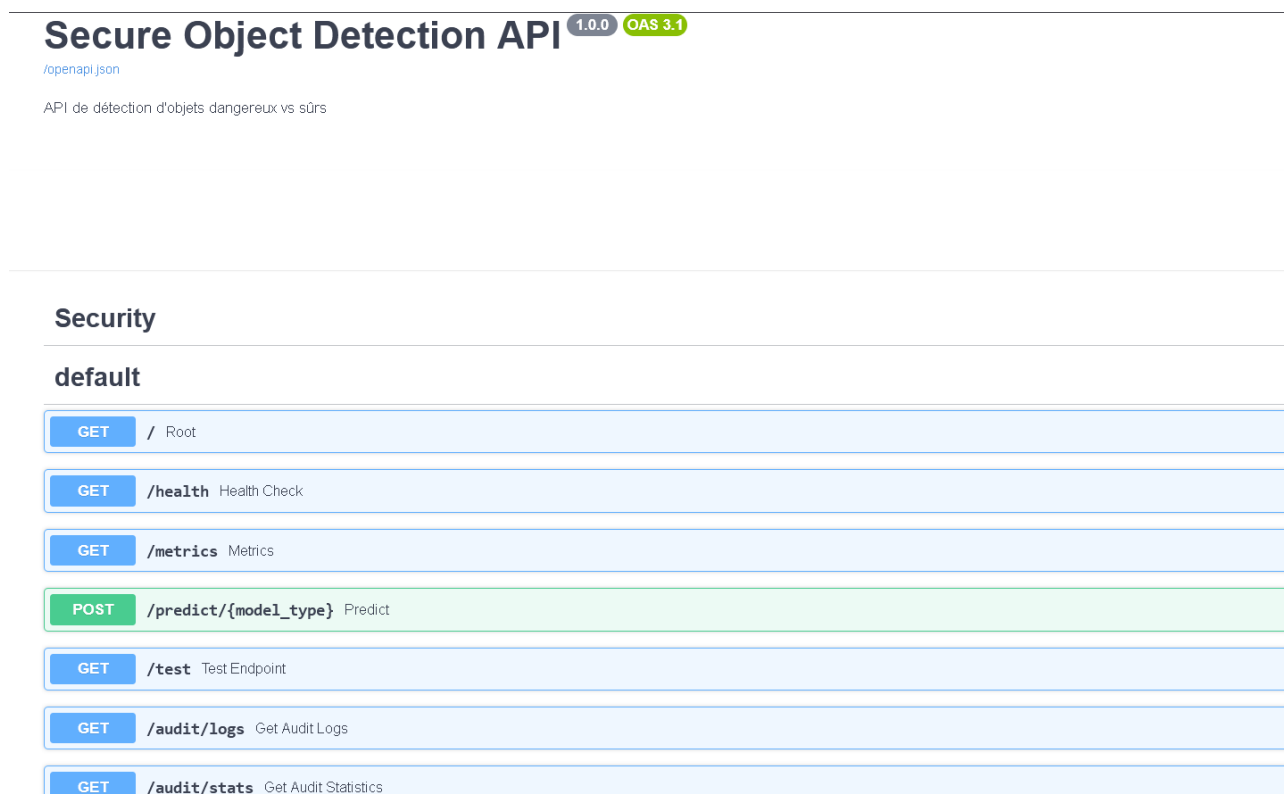


FIGURE 4.6 – Documentation API Swagger : endpoints disponibles

L'API expose 20+ endpoints répartis en 4 catégories : prédiction, sécurité, audit, et administration.

4.1.3 Dashboards de monitoring Grafana

Grafana fournit des dashboards en temps réel pour le monitoring de l'API et de la sécurité. La figure 4.7 montre le dashboard principal.

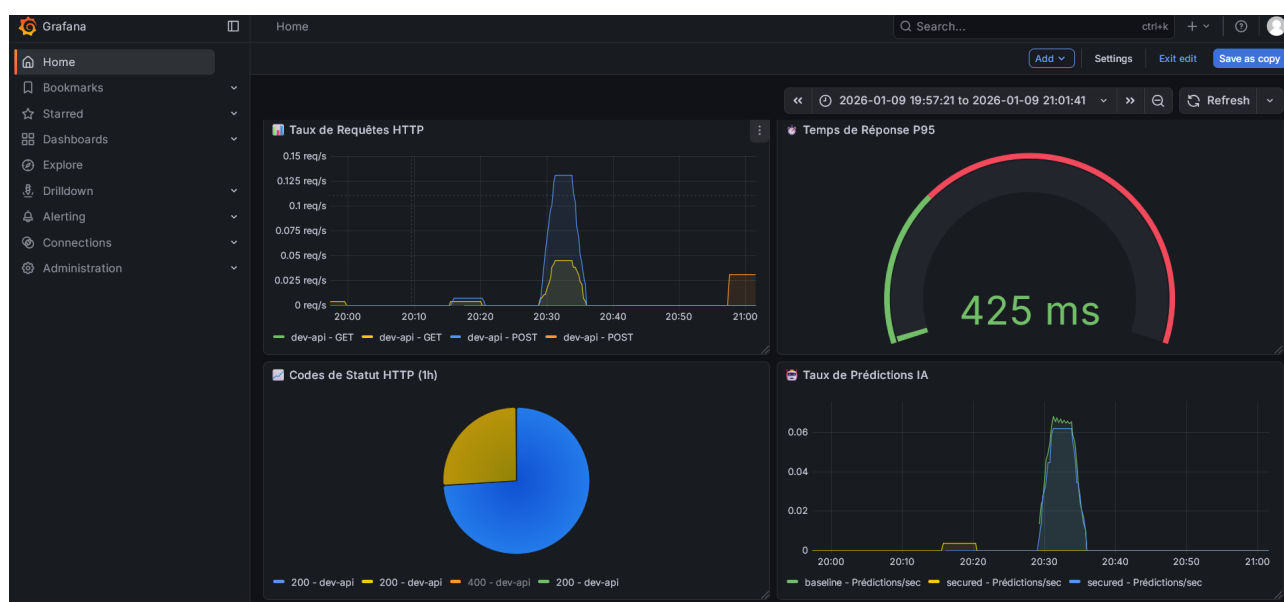


FIGURE 4.7 – Dashboard Grafana : métriques en temps réel de l'API

4.2 Discussions

Notre structure à cinq couches se démarque par son approche globale qui intègre simultanément diverses dimensions de sécurité :

- **Validation des données d'entrée (Couche 1)** : Vérification statistique des données, contrôle de données corrompues.
- **Robustesse adversariale (Couche 2)** : Entraînement adversarial (FGSM) avec renforcement de la robustesse
- **Validation de sortie (Couche 3)** : Identification des anomalies par scoring du risque et seuils adaptatifs.
- **Sécurisation des accès (Couche 4)** : Authentification JWT avec RBAC, WAF incluant la détection d'injection SQL et XSS, limitation de fréquence (req/min), audit immuable avec hash chains
- **Gouvernance (Couche 5)** : Surveillance en temps réel (Prometheus/Grafana), tableaux de bord de sécurité, système d'alerte.

4.2.1 Comparaison avec l'état de l'art

Le tableau 4.4 compare notre architecture à 5 couches avec les principales approches existantes en sécurisation de systèmes ML.

TABLE 4.4 – Analyse comparative des principales approches de sécurisation des systèmes d'IA

Critères d'évaluation	Madry [20]	TRADES	NIST AI RMF	Google SAIF	Microsoft AI Security	Architecture proposée
Robustesse face aux attaques adversariales	+++	+++	–	+	+	+++
Protection des données d'entraînement	–	–	+	++	++	+++
Intégrité des modèles	–	–	–	+	++	+++
Sécurisation des accès et des API	–	–	–	+	+++	+++
Détection d'anomalies	–	–	–	+	++	++
Audit et traçabilité	–	–	+++	++	++	+++
Surveillance continue	–	–	+	++	++	+++
Intégration DevSecOps	–	–	+	++	++	+++
Déploiement opérationnel	Faible	Faible	Conceptuel	Moyen	Élevé	Élevé
Validation expérimentale	+++	+++	–	–	+	+++
Couverture du cycle de vie IA	Faible	Faible	Moyenne	Élevée	Élevée	Très élevée

(+++)
(++) Couverture forte (++) Couverture moyenne (+) Couverture limitée (–) Non couverte

TABLE 4.5 – Réponse de l’architecture proposée aux limites identifiées dans l’état de l’art

Limites identifiées dans l’état de l’art	Réponse apportée par l’architecture proposée
Protection généralement limitée à la robustesse du modèle	Architecture multicouche intégrant simultanément la protection des données, la robustesse du modèle et la sécurité de l’infrastructure.
Faible prise en compte de la sécurité des modèles déployés	Chiffrement, signature et vérification d’intégrité avant chargement du modèle.
Absence de mécanismes de contrôle d’accès dédiés aux systèmes IA	Mise en œuvre d’une authentification JWT, d’un contrôle RBAC et d’un WAF protégeant les services exposés.
Faible couverture de la détection d’activités malveillantes en phase d’exploitation	Détection d’anomalies basée sur le scoring de risque et l’analyse comportementale des requêtes.
Manque de supervision continue et de traçabilité	Intégration de Prometheus, Grafana et de journaux d’audit immuables pour assurer le suivi des opérations.
Approches souvent limitées à une phase du cycle de vie IA	Couverture des phases de préparation des données, entraînement, déploiement, exploitation et supervision continue.
Faible validation pratique des frameworks existants	Implémentation complète et validation expérimentale sur un système de vision par ordinateur basé sur MobileNetV2.

4.2.2 Positionnement de notre contribution

Notre travail se distingue des approches existantes par quatre contributions principales :

1. **Intégration holistique** : Au lieu de se focaliser sur un seul aspect (soit la robustesse face aux attaques, soit la confidentialité, soit le DevSecOps), notre structure à cinq couches intègre conjointement la défense contre les attaques, la vérification multicouche, la sécurisation des accès et la gouvernance complète.
2. **Adaptation au contexte opérationnel** : À l’inverse des frameworks génériques (SAIF), notre architecture est spécifiquement conçue pour identifier des images nuisibles avec des exigences de temps réel (<100 ms), un équilibre solide entre robustesse et accuracy (93.1% clean, 68.1% FGSM), une traçabilité intégrale grâce à un audit inaltérable et un déploiement en conteneurs hautement disponible.
3. **Validation empirique complète** : À l’opposé des frameworks conceptuels tels que le NIST AI RMF, nous offrons une mise en œuvre tangible et une évaluation quantitative de la performance

de chaque zone sécurisée, mesurant les indicateurs réels sur un ensemble de données de plus de 1000 images.

4. **Approche pragmatique et extensible** : Architecture modulaire autorisant l'intégration progressive de défenses additionnelles (PGD training, défenses certifiées, confidentialité différentielle) sans nécessiter une refonte intégrale du système.

4.2.3 Limites identifiées

Malgré les résultats prometteurs, notre approche présente certaines limitations qui doivent être prises en compte :

1. **Démonstration de robustesse limitée aux attaques FGSM** : L'entraînement adversarial, qui lors de l'expérimentation se concentre sur les attaques FGSM ($= 0.08$), indique une robustesse renforcée, bien qu'il ne soit pas suffisamment généralisable. Cela entraîne une diminution de la précision de 8,16 % sur les données propres et demeure restreint à un epsilon fixe, ne prenant en compte qu'une fraction des perturbations possibles.
2. **Dataset de taille limitée** : Le dataset d'entraînement final (998 images après quarantaine) est restreint. Un dataset plus large (10k+ images) permettrait d'améliorer la généralisation et la robustesse.
3. **Tests limités en conditions réelles** : Les tests ont été effectués en environnement contrôlé (dataset statique). Une évaluation sur du trafic réel avec attaquants adversariaux adaptatifs serait nécessaire pour valider l'efficacité en production.

4.2.4 Perspectives d'amélioration

Les limites identifiées ouvrent plusieurs perspectives de recherche et d'amélioration :

- **Adversarial training multi-attaques** : Entraîner en combinant FGSM, PGD, et C&W pour renforcer la généralisation de la robustesse face aux attaques adversariales : 50% clean, 25% FGSM, 15% PGD, 10% C&W .
- **Tests d'intrusion automatisés continus** : Déployer un framework d'attaques continues (AutoAttack, RobustBench) en CI/CD pour valider la robustesse à chaque modification du modèle et détecter les régressions.

Conclusion

Ce chapitre a exposé les résultats expérimentaux de notre architecture en 5 couches à travers un système de détection d'objets dangereux. L'architecture multicouche de Docker a rendu possible le

déploiement réussi de huit services interconnectés pour le développement et la production. Le modèle entraîné avec la technique adversariale FGSM a démontré une nette amélioration en termes de robustesse face aux attaques adversariales FGSM, tout en maintenant une précision satisfaisante sur les données non altérées. Les interfaces, l'API et les tableaux de bord démontrent la faisabilité d'un déploiement en production.

Toutefois, des limites demeurent : le modèle reste exclusivement formé sur FGSM et le jeu de données utilisé demeure limité privilégie la vitesse au détriment d'une robustesse optimale. Des améliorations ultérieures sont prévues pour accroître la robustesse et transformer cette architecture en une solution de sécurité IA plus exhaustive et fiable.

Conclusion Générale

Ce mémoire a souligné l'importance cruciale de sécuriser les systèmes d'intelligence artificielle dans les dispositifs de grande sensibilité, compte tenu d'un spectre étendu de vulnérabilités, allant des manipulations de données aux failles structurelles de l'infrastructure. Afin de répondre à ces défis, nous avons élaboré une architecture de sécurité multicouche en cinq couches, basée sur les principes du DevSecOps et de la défense en profondeur. Cette approche va au-delà de la simple protection algorithmique et comprend une validation minutieuse des données d'entrée, la sécurisation des communications et le renforcement de l'environnement de production. Cela garantit ainsi une robustesse systémique du modèle face aux attaques d'empoisonnement, d'évasion ou à l'exploitation de failles logicielles.

La mise en œuvre de cette stratégie, via notre système de détection d'objets dangereux, illustre l'efficacité d'une panoplie défensive variée. En conjuguant l'entraînement proactif, le chiffrement des communications, la signature des modèles ainsi qu'un monitoring proactif grâce à Prometheus et Grafana, nous avons démontré que la sécurité robuste ne s'appuie pas sur une seule mesure, mais sur l'harmonie entre les contrôles techniques et opérationnels.

Bien que l'automatisation de la réponse aux incidents et le recours à la confidentialité différentielle offrent des perspectives, ce travail démontre que la fiabilité de l'IA sécuritaire repose sur une approche globale où l'innovation technologique ne peut être dissociée d'une architecture défensive stricte et en constante évolution.

Bibliographie

- [1] APDP - bénin. <https://service.apdp.bj/documentations/lois/352/1>.
- [2] Attacking artificial intelligence : AI's security vulnerability and what policymakers can do about it | the belfer center for science and international affairs. <https://www.belfercenter.org/publication/AttackingAI>.
- [3] Introduction à l'apprentissage automatique - CultureSciences de l'ingénieur - éducol STI. Consulté à l'adresse https://sti.eduscol.education.fr/si-ens-paris-saclay/ressources_pedagogiques/introduction-a-lapprentissage-automatique.
- [4] ISO/IEC 42001 :2023.
- [5] Proposition de RÈGLEMENT DU PARLEMENT EUROPÉEN ET DU CONSEIL ÉTABLISSANT DES RÈGLES HARMONISÉES CONCERNANT L'INTELLIGENCE ARTIFICIELLE (LÉGISLATION SUR L'INTELLIGENCE ARTIFICIELLE) ET MODIFIANT CERTAINS ACTES LÉGISLATIFS DE L'UNION. <https://eur-lex.europa.eu/legal-content/FR/TXT/?uri=CELEX%3A52021PC0206>.
- [6] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 308–318.
- [7] N. Akhtar and A. Mian. Threat of adversarial attacks on deep learning in computer vision : A survey.
- [8] D. Andre. Qu'est-ce qu'un modèle? <https://www.allaboutai.com/fr-fr/glossaire-ai/modele/>.
- [9] P. Besse, A. Besse-Patin, and C. Castets-Renard. Implications juridiques et éthiques des algorithmes d'intelligence artificielle dans le domaine de la santé. (8). Publisher : Société française de statistique (SFdS).
- [10] B. Biggio, B. Nelson, and P. Laskov. Poisoning attacks against support vector machines.
- [11] N. Carlini, C. Liu, [U+FFFD] Erlingsson, J. Kos, and D. Song. The secret sharer : Evaluating and testing unintended memorization in neural networks.
- [12] A. Esteva, B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun. Dermatologist-level classification of skin cancer with deep neural networks. 542(7639) :115–118. Publisher : Nature Publishing Group.

-
- [13] S. Fazelpour and D. Danks. Algorithmic bias : Senses, sources, solutions. 16(8) :e12760.
- [14] C. Gentry. A fully homomorphic encryption scheme, 2009.
- [15] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples.
- [16] Google Cloud. Secure ai framework (saif) : A conceptual framework for secure ai systems. Technical report, Google LLC, 2023.
- [17] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao. Advances and open problems in federated learning.
- [18] kenwith. Securing the future of AI and ML at microsoft, 2020.
- [19] Z. C. Lipton. The mythos of model interpretability.
- [20] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. Towards deep learning models resistant to adversarial attacks.
- [21] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan. A survey on bias and fairness in machine learning. 54(6) :115 :1–115 :35.
- [22] National Institute of Standards and Technology. AI risk management framework. Last Modified : 2025-05-05T13 :16-04 :00.
- [23] M.-I. Nicolae, M. Sinn, M. N. Tran, B. Buesser, A. Rawat, M. Wistuba, V. Zantedeschi, N. Baracaldo, B. Chen, H. Ludwig, I. M. Molloy, and B. Edwards. Adversarial robustness toolbox v1.0.0.
- [24] D. Oliynyk, R. Mayer, and A. Rauber. I know what you trained last summer : A survey on stealing machine learning models and defences. 55(14) :324 :1–324 :41.
- [25] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami. Distillation as a defense to adversarial perturbations against deep neural networks.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch : An imperative style, high-performance deep learning library.
- [27] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. MobileNetV2 : Inverted residuals and linear bottlenecks.
- [28] R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models.

-
- [29] J. Steinhardt, P. W. Koh, and P. Liang. Certified defenses for data poisoning attacks.
- [30] S. Thys, W. Van Ranst, and T. Goedemé. Fooling automated surveillance cameras : Adversarial patches to attack person detection. abs/1904.08653. arXiv preprint arXiv :1904.08653 [cs.CV].
- [31] P. Zaraté. L'intelligence artificielle d'hier à aujourd'hui. *Droit Social, Dalloz, C*, 2, 2021.
- [32] H. Zhang, Y. Yu, J. Jiao, E. P. Xing, L. E. Ghaoui, and M. I. Jordan. Theoretically principled trade-off between robustness and accuracy.

Annexe

Description des couches du cas expérimental

Dans le cadre du cas expérimental, l'architecture proposée est mise en œuvre sous la forme d'un système organisé en cinq couches fonctionnelles interconnectées. Chaque couche représente un ensemble de composants logiciels déployés pour garantir le bon fonctionnement, la sécurité et la robustesse du modèle d'IA tout au long de son cycle d'exploitation. La figure 4.8 illustre cette organisation modulaire.

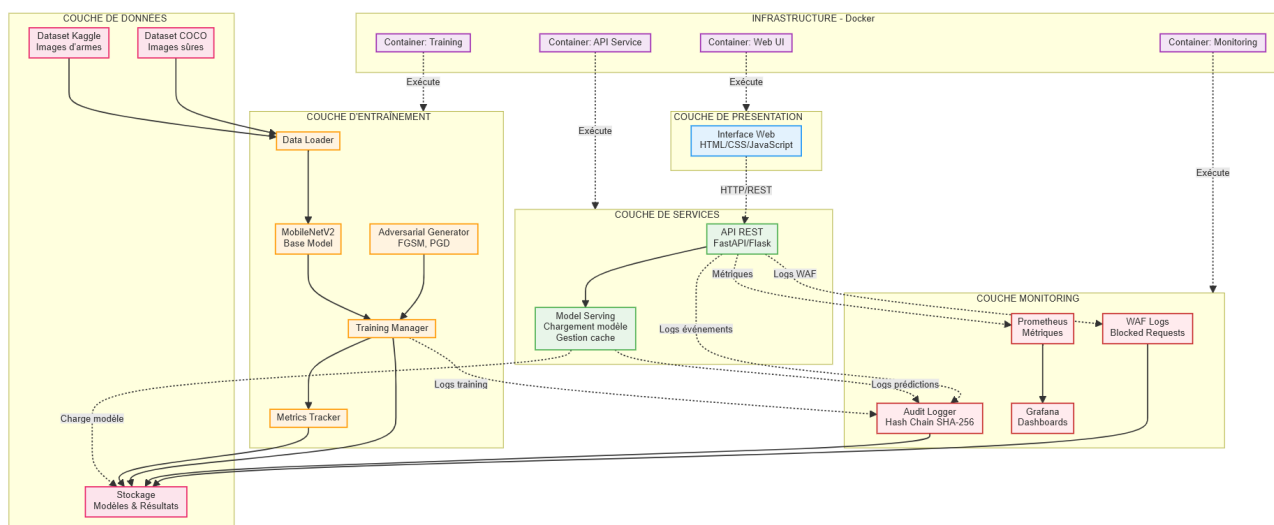


FIGURE 4.8 – Description des couches du système.

La **couche de données** s'occupe du chargement, du prétraitement et de la préparation du jeu de données qui a servi à l'entraînement et à l'évaluation du modèle. Elle est mise en œuvre à travers des scripts Python qui s'occupent de la normalisation des images, de la gestion des jeux d'apprentissage et de test, ainsi que des vérifications d'intégrité destinées à limiter les risques liés à l'introduction de données corrompues ou malveillantes.

La **couche d'entraînement** met en œuvre le processus d'apprentissage du modèle MobileNetV2 dans un environnement isolé. Cette couche intègre un mécanisme d'apprentissage adversarial basé sur l'attaque FGSM, qui permet la création d'exemples adverses pendant la phase d'entraînement. On surveille les indicateurs de performance et de robustesse du modèle pour juger de sa réaction aux perturbations intentionnelles.

La **couche de services** est implémentée sous la forme d’une API REST exposant les fonctionnalités d’inférence du modèle entraîné. Cette couche permet l’analyse de nouvelles images, l’évaluation de la robustesse du modèle en production et l’accès contrôlé aux résultats. Des dispositifs de protection de l’API sont instaurés pour restreindre les accès non autorisés et minimiser les risques d’utilisation abusive du service.

La **couche de présentation** offre une interface web permettant une interaction directe avec l’utilisateur. Elle sert à la soumission d’images, l’affichage des résultats d’inférence et présentation des indicateurs de performance du modèle. Cette couche rend l’utilisation du système expérimental plus facile et la compréhension des résultats obtenus plus claire.

Pour finir, la **couche infrastructure** assure le déploiement, l’exécution et la disponibilité de l’ensemble des services du système expérimental. Elle repose sur une infrastructure conteneurisée basée sur Docker, permettant l’orchestration des différents services, la gestion des ressources matérielles et l’isolation des environnements.

Évaluation des vulnérabilités du modèle de base

Cette partie présente la mise en œuvre de l’approche baseline qui consiste en l’évaluation de la robustesse face aux attaques adversariales du modèle de référence MobileNetV2 qui a été entraîné sur notre jeu de données. L’objectif est de repérer les faiblesses du modèle face aux attaques adversariales avant de mettre en place des stratégies de défense.

Méthodologie d’évaluation

Le module `attack_baseline.py` intègre l’évaluation des vulnérabilités, reposant sur une structure de test sécurisée. L’évaluation est effectuée en sandbox isolée de manière systématique afin d’assurer la reproductibilité des tests.

Organisation du code de test La classe `AdversarialTester` contient le testeur adversarial, qui prend en charge le chargement sécurisé du modèle, la création d’attaques et l’enregistrement des résultats. Cette méthode modulaire facilite la séparation des tests adversariaux du reste de l’appareil et garantit un suivi intégral des expérimentations grâce à des journaux inaltérables.

Configuration des tests Les tests sont configurés avec les paramètres suivants :

- **Modèle testé** : MobileNetV2 baseline (97.5% accuracy propre)
- **Dataset de test** : 204 échantillons équilibrés (102 safe, 102 dangerous)
- **Batch size** : 1 (tests adversariaux précis par échantillon)
- **Device** : CPU (reproductibilité et isolation de sécurité)

La classe `AdversarialTester` initialise le système de logging avant tout autre opération, garantissant la traçabilité complète de chaque test adversarial :

```

1 class AdversarialTester:
2     def __init__(self, model_path, test_data_path, device='cpu'):
3         self.device = torch.device(device)
4         self.security_log = []
5         self._log_security_event("INIT",
6                                 "Initialisation du testeur adversarial", "INFO")
7
8         # Chargement du modele en sandbox isolee
9         self.model = self._load_model()
10        self.test_loader = self._load_test_data()

```

Listing 4.1 – Initialisation du testeur adversarial

Ce mécanisme de logging consignent chaque incident en notant un horodatage, une catégorie d’incident et un degré de sévérité, offrant ainsi un audit exhaustif des tests effectués.

Implémentation de l’attaque FGSM

Pour cette évaluation initiale, nous avons opté pour l’attaque FGSM (Méthode du Signe du Gradient Rapide), car elle constitue l’attaque adversariale de référence la plus simple et rapide à mettre en œuvre. La mise en œuvre suit la formulation classique de [15] : $x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$, où ϵ contrôle la magnitude de la perturbation.

L’algorithme FGSM est implémenté dans la méthode `fgsm_attack()` avec les étapes suivantes :

```

1 def fgsm_attack(self, epsilon=0.1):
2     self.model.eval()
3
4     for data, target in self.test_loader:
5         data.requires_grad = True
6
7         # Prediction propre
8         output_clean = self.model(data)
9         pred_clean = output_clean.argmax(dim=1)
10
11        # Calcul du gradient
12        loss = F.cross_entropy(output_clean, target)
13        self.model.zero_grad()
14        loss.backward()
15
16        # Generation FGSM
17        data_grad = data.grad.data
18        perturbed_data = data + epsilon * data_grad.sign()
19        perturbed_data = torch.clamp(perturbed_data, 0, 1)
20
21        # Test adversarial
22        output_adversarial = self.model(perturbed_data)
23        pred_adversarial = output_adversarial.argmax(dim=1)

```

Listing 4.2 – Génération d’exemples adverses FGSM

Après normalisation, la perturbation est limitée à l'intervalle $[0, 1]$ afin de s'assurer que des images adversariales sont valides. Le choix de la valeur de $\epsilon = 0.1$ et été fait afin simuler une perturbation assez discrète, mais assez forte pour évaluer la robustesse du modèle.

Métriques calculées Lors de chaque test, le système évalue quatre métriques principales stockées au format JSON :

- **Clean accuracy** : Performance sur images propres non altérés
- **Adversarial accuracy** : Performance sur images adversariales
- **Attack success rate** : Taux de réussite de l'attaque
- **Robustness degradation** : Écart entre accuracy propre et adversariale

Résultats et identification des failles

L'évaluation du modèle baseline révèle une vulnérabilité significative face à l'attaque FGSM. Le fichier de résultats `fgsm_results_20251229_133702.json` contient les métriques suivantes :

TABLE 4.6 – Résultats de l'attaque FGSM sur le modèle baseline

Métrique	Valeur	Interprétation
Échantillons testés	204	Ensemble de test complet
Clean accuracy	97.5%	Excellent sur données propres
Adversarial accuracy	50.0%	Dégradation critique
Attack success rate	50.0%	Vulnérabilité élevée
Robustness degradation	47.5%	Perte importante
Epsilon utilisé	0.1	Perturbation faible

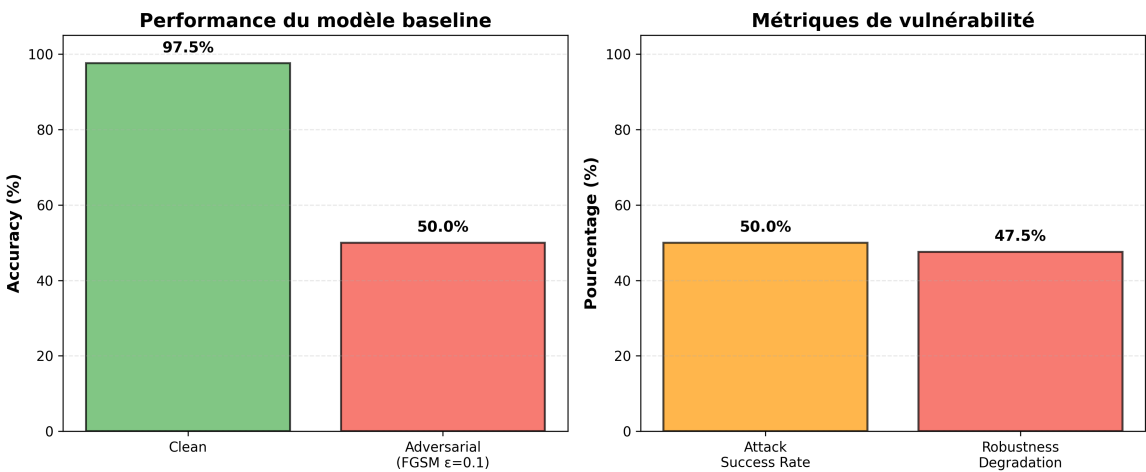


FIGURE 4.9 – Comparaison de la performance baseline : données propres vs adversariales FGSM

Analyse des vulnérabilités identifiées :

Les résultats mettent en évidence trois failles majeures du modèle baseline :

1. Sensibilité critique aux perturbations de gradient. Avec une détérioration de 47.5%, le modèle perd presque la moitié de sa capacité à classer lorsqu'il est confronté à des perturbations calculées par gradient. Cette réactivité est due à l'absence de dispositifs de robustesse lors de l'entraînement traditionnel : le modèle ne cherche qu'à maximiser la performance sur des données propres sans tenir compte des variations adversariales.

2. Vulnérabilité symétrique sur les deux classes. L'attack success rate de 50% signifie que la vulnérabilité ne se limite pas à une classe particulière (safe ou dangerous), mais touche les deux classes de façon équilibrée. Cela indique que le modèle fonde ses décisions sur des caractéristiques fragiles facilement manipulables par gradient descent, plutôt que sur des patterns robustes.

3. Absence de détection d'anomalies. Le modèle ne possède pas de système pour identifier les entrées adversariales. Les exemples altérés sont gérés avec le même degré de confiance que les images naturelles, sans aucune alerte ni refus. Ce manquement représente un danger de sécurité important lors d'une mise en production, où un ennemi pourrait tirer parti de cette faille pour camoufler une arme en tant qu'objet inoffensif.

Implications pour la sécurisation : Au vue de ces résultats, dans un contexte de sécurité publique, une telle dégradation pourrait conduire à la non-détection d'un objet dangereux ou à une fausse alerte. Il est donc nécessaire d'intégrer des défenses adversariales dans le processus d'entraînement. Le pipeline sécurisé intégrant l'entraînement adversarial pour minimiser considérablement ces vulnérabilités sera présenté dans la section suivante (3.4). L'ambition est de parvenir à une précision face au FGSM supérieure à 80%, diminuant de ce fait l'écart de robustesse en dessous des 20%.

Structure des fichiers Docker de développement

La structure de l'environnement de développement suit l'organisation suivante :

```
docker/
|-- Dockerfile.dev           Image de développement (Python 3.10)
|-- Dockerfile.test         Image pour les tests automatisés
|-- Dockerfile.web          Image Nginx pour l'interface web
`-- docker-compose.dev.yml   Orchestration des 8 services

requirements.txt             Dépendances Python de production
requirements-dev.txt         Dépendances de développement
```

Configuration Docker Compose de développement

Les 8 services de l'environnement dev sont orchestrés par le fichier `docker-compose.dev.yml` à travers un réseau bridge dédié (`secure-ai-network`).

```
1 services:
2   dev:
3     build:
4       context: ..
5       dockerfile: docker/Dockerfile.dev
6     container_name: secure-ai-dev
```

```

7  test:
8    build:
9      context: ..
10     dockerfile: docker/Dockerfile.test
11     container_name: secure-ai-test
12  postgres:
13    image: postgres:15-alpine
14    container_name: secure-ai-db
15  redis:
16    image: redis:7-alpine
17    container_name: secure-ai-redis
18  prometheus:
19    image: prom/prometheus:latest
20    container_name: secure-ai-prometheus
21  grafana:
22    image: grafana/grafana:latest
23    container_name: secure-ai-grafana
24  node-exporter:
25    image: prom/node-exporter:latest
26    container_name: secure-ai-node-exporter
27  web:
28    build:
29      context: ..
30      dockerfile: docker/Dockerfile.web
31    container_name: secure-ai-web

```

Listing 4.3 – docker-compose.dev.yml (extrait)

Les Dépendances Python pour le développement

L'environnement de développement nécessite deux fichiers de dépendances : `requirements.txt` pour les bibliothèques de base (PyTorch, FastAPI, sécurité) et `requirements-dev.txt` pour les outils de développement avancés. L'installation se fait via `pip install -r requirements.txt -r requirements-dev.txt` dans le conteneur dev.

Configuration Docker Compose de l'environnement prod

```

1  services:
2    # API Securisee
3    secured-api:
4      build:
5        context: ..
6        dockerfile: docker/Dockerfile.secured
7    # API Baseline
8    baseline-api:
9      build:
10       context: ..

```

```
11     dockerfile: docker/Dockerfile.baseline
12     container_name: baseline-apil
13     # Nginx Load Balancer + Web Server
14     nginx:
15         image: nginx:alpine
16         container_name: secure-ai-nginx1
17     # Interface Web (sur port séparé)
18     web:
19         image: nginx:alpine
20         container_name: secure-ai-web-prod1
21     # Base de données PostgreSQL
22     postgres:
23         image: postgres:15-alpine
24         container_name: secure-ai-db-prod1
25     # Cache Redis
26     redis:
27         image: redis:7-alpine
28         container_name: secure-ai-redis-prod1
29     # Monitoring Prometheus
30     prometheus:
31         image: prom/prometheus:latest
32         container_name: prometheus-prod1
33     # Monitoring Grafana
34     grafana:
35         image: grafana/grafana:latest
36         container_name: grafana-prod1
```

Listing 4.4 – docker-compose.prod.yml (extrait)

Table des matières

Dédicace	ii
Remerciements	iii
Résumé	iv
Abstract	v
List of Figures	vi
List of Tables	vii
Liste des acronymes	viii
Introduction	1
1 Etat de l'art	4
Introduction	4
1.1 L'intelligence artificielle et ses applications	4
1.1.1 Notion d'intelligence artificielle	4
1.1.2 Importance et impact de l'IA dans les systèmes modernes	5
1.1.2.1 La Santé et la Médecine de Précision	5
1.1.2.2 Le Transport et les Véhicules Autonomes	5
1.1.2.3 Le Secteur Financier, Bancaire et l'Économie Globale	6
1.1.2.4 La Relation Client, les Services et la Gestion des Flux d'Information	6
1.1.2.5 La Gestion des Infrastructures Critiques et de l'Énergie	6
1.1.2.6 La Sécurité Publique, la Télésurveillance et l'Analyse en Milieu Sensible	7
1.1.2.7 Le Domaine Juridique et l'Aide à la Décision Institutionnelle	7
1.1.3 Notion de systèmes d'IA critiques	8
1.2 Sécurité et intelligence artificielle	8
1.3 Typologie des vulnérabilités des modèles d'IA	9
1.3.1 Vulnérabilités structurelles et inhérentes aux modèles IA	9
1.3.2 Vulnérabilités liées aux données d'entraînement	9
1.3.2.1 Empoisonnement des données (Data Poisoning) :	10
1.3.2.2 Biais et discrimination algorithmique :	10
1.3.2.3 Fuites d'informations sensibles :	10
1.3.3 Vulnérabilités liées à l'inférence et à l'exploitation des modèles	10

1.3.3.1	Vulnérabilités par inférence de modèle :	10
1.3.3.2	Vulnérabilités par extraction de modèle	11
1.4	Classification des attaques contre les modèles d'IA	11
1.4.1	Attaques adversariales	11
1.4.2	Attaques sur les données d'entraînement	12
1.4.2.1	Poisoning Attacks (attaques d'empoisonnement)	12
1.4.2.2	Backdoor Attacks (attaques par porte dérobée)	12
1.4.3	Attaques sur la confidentialité et la robustesse des modèles	12
1.4.3.1	Membership Inference Attack (MIA)	13
1.4.3.2	Model Extraction Attacks :	13
1.4.3.3	Illustration de l'évolution des menaces et des défenses	13
1.5	Analyse des travaux existants sur la protection des systèmes IA	15
1.5.1	Approches de défense adversariale	15
1.5.1.1	Entraînement adversarial	15
	Formalisation Mathématique :	15
	Résultats Opérationnels et Robustesse :	15
	Limites :	16
1.5.1.2	TRADES et défenses basées sur la régularisation	16
	Apport Spécifique :	16
	Limites Techniques :	16
1.5.1.3	Distillation défensive et transformations d'entrée	16
	Résultats Opérationnels :	17
	Limites :	17
1.5.2	Techniques de préservation de la confidentialité	17
1.5.2.1	Confidentialité différentielle	18
	Apports :	18
	Limites :	18
1.5.2.2	Apprentissage fédéré	18
	Apports :	18
	Limites :	18
1.5.2.3	Chiffrement homomorphe	18
	Apports :	18
	Limites :	19
1.5.3	Frameworks et architectures de sécurité ML	19
1.5.3.1	NIST AI Risk Management Framework (AI RMF)	19
	Apports :	19
	Limites :	19
1.5.3.2	Secure AI Framework (SAIF)	19
	Apports :	19
	Limites :	19
1.5.3.3	Microsoft AI Security Architecture	19
	Apports :	20
	Limites :	20
1.5.4	Synthèse critique et positionnement de nos travaux	20
1.6	Éthique et cadre réglementaire des modèles IA	21

1.6.1	Principes fondamentaux de l'éthique en IA	21
1.6.2	Régulations et normes	21
2	Méthodologie et Architecture	23
	Introduction	23
2.1	Démarche méthodologique	23
2.1.1	Collecte des Données	23
2.1.2	Modélisation d'une Architecture de sécurité	23
2.1.3	Mise en place d'un cas expérimental	24
2.1.4	Simulation et Évaluation des Résultats	24
2.2	Méthode de sécurisation des systèmes IA dans les environnements critiques	24
2.2.1	Architecture globale de protection des systèmes IA	24
2.2.1.1	Architecture en cinq couches de sécurité	25
2.2.2	Approche DevSecOps dans la protection des systèmes IA.	30
2.2.3	Recommandations éthiques des systèmes d'IA	30
2.3	Outils et Technologies	31
2.3.1	Environnement de développement	31
	Python :	31
	PyTorch :	31
	Google Colab / Jupyter Notebook :	31
2.3.2	Bibliothèques de sécurité et de robustesse	31
	Adversarial Robustness Toolbox (ART) :	31
	Foolbox :	32
2.3.3	Infrastructure de déploiement	32
	Docker :	32
	FastAPI :	32
	Nginx :	32
2.3.4	Monitoring et observabilité	32
	Prometheus + Grafana :	32
	ELK Stack (Elasticsearch, Logstash, Kibana) :	32
2.3.5	Justification des choix technologiques	33
3	Implémentation de l'approche proposée	34
	Introduction	34
3.1	Présentation du cas expérimental	34
3.1.1	Objectif et scénario de l'expérimentation	34
3.1.2	Présentation du dataset	35
3.2	Choix du modèle de base	36
3.3	Développement et entraînement sécurisé	36
3.3.1	Environnement de développement Docker	36
	Description des 8 services	37
3.3.2	Pipeline d'entraînement sécurisé	37
3.3.2.1	Sécurisation des données pré-entraînement	39
3.3.2.2	Adversarial Training avec FGSM	39
3.3.2.3	Configuration d'entraînement	40

3.3.2.4	Sauvegarde des checkpoints d'entraînement	41
3.3.2.5	Chiffrement AES-256-GCM du modèle	41
3.3.2.6	Signature RSA-4096 pour non-répudiation	42
3.3.3	API de développement et interface web	42
3.3.3.1	Chargement sécurisé du modèle au démarrage	42
3.3.3.2	Endpoints de l'API de développement	43
3.3.3.3	Interface web de développement	43
3.4	Architecture et déploiement en production	43
3.4.1	Architecture Docker de production	43
	Stack Docker de production	44
3.4.2	Sécurisation en production	45
	Web Application Firewall (WAF)	45
	Authentification JWT (JSON Web Token) et contrôle d'accès RBAC (Role-Based Access Control)	45
	Détection d'anomalies comportementales	45
3.4.3	Gouvernance et audit	46
	Logs d'audit immuables	46
	Monitoring temps réel avec Prometheus et Grafana	46
Conclusion		47
4	Résultats et discussion	48
	Introduction	48
4.1	Résultats obtenus	48
4.1.1	Résultats d'entraînement du modèle sécurisé	48
4.1.2	Interfaces utilisateur et API	51
4.1.3	Dashboards de monitoring Grafana	53
4.2	Discussions	54
4.2.1	Comparaison avec l'état de l'art	54
4.2.2	Positionnement de notre contribution	55
4.2.3	Limites identifiées	56
4.2.4	Perspectives d'amélioration	56
Conclusion		56
Conclusion		58
Bibliographie		59
Bibliographie		59
Annexe		62
	Organisation du code de test	63
	Configuration des tests	63
	Métriques calculées	65
	Analyse des vulnérabilités identifiées	65
	Implications pour la sécurisation :	66
	Configuration Docker Compose de développement	66

Les Dépendances Python pour le développement	67
Table des matières	69
